
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский физико-технический институт
(национальный исследовательский университет)»
Физтех-школа Радиотехники и Компьютерных Технологий
Кафедра интеллектуальных информационных систем и технологий

Направление подготовки / специальность: 03.03.01 Прикладные математика и физика

Направленность (профиль) подготовки: Радиотехника и компьютерные технологии

ИССЛЕДОВАНИЕ МЕТОДОВ ОПРЕДЕЛЕНИЯ ХАРАКТЕРИСТИК ТРАНСПОРТНОГО СРЕДСТВА ПО ВИДЕОИЗОБРАЖЕНИЯМ

(бакалаврская работа)

Студент:

Клименко Виталий Евгеньевич

(подпись студента)

Научный руководитель:

Назаров Алексей Николаевич,
д-р техн. наук, проф.

(подпись научного руководителя)

Консультант (при наличии):

(подпись консультанта)

Москва 2026

Аннотация

Ключевые слова: детекция транспортных средств, распознавание номерных знаков, классификация цвета и типа кузова, YOLO26, ResNet18, EasyOCR, transfer learning, глубокое обучение.

Цель работы — разработка системы автоматического определения характеристик транспортных средств (номерной знак, цвет, тип кузова) по видеопотоку.

Задачи: анализ методов детекции и классификации, выбор и обоснование архитектур глубокого обучения, подготовка датасетов, обучение и оценка моделей, интеграция в единый конвейер обработки видео.

Результаты: предложена конвейерная архитектура (YOLO26n — детекция ТС и номеров, ResNet18 — цвет и тип кузова, EasyOCR — OCR кириллицы). Детектор номеров достиг $mAP@0.50 = 0.9669$; классификатор цвета — $Accuracy = 0.936$; типа кузова — $Accuracy = 0.830$; EasyOCR — $CRR = 0.750$ на реальных данных.

Рекомендации: заменить EasyOCR специализированным OCR для российских номеров и расширить набор классов атрибутов.

Аннотация	1
Обозначения и сокращения	4
Введение	5
Актуальность задачи	5
Цель и задачи работы	5
Постановка задачи	5
1 Обзор существующих методов	7
1.1 Детекция транспортных средств	7
1.1.1 Сравнение подходов к детекции	7
1.1.2 Двухэтапные детекторы (Faster R-CNN)	7
1.1.3 Одноэтапные детекторы (YOLO)	7
1.2 Распознавание номерных знаков (ANPR)	7
1.2.1 Сравнение ANPR-подходов	8
1.3 Распознавание марки и типа кузова (VMMR)	8
1.4 Определение цвета автомобиля	8
2 Описание выбранных методов и датасетов	10
2.1 Архитектура системы	10
2.2 Теоретические основы глубокого обучения	10
2.2.1 Сверточные нейронные сети	10
2.2.2 Функции активации	11
2.2.3 Pooling и агрегация признаков	11
2.2.4 Пакетная нормализация	12
2.3 Детекция транспортных средств: YOLO26	12
2.3.1 Внутренняя архитектура YOLO26	12
2.3.2 Эволюция семейства YOLO	13
2.3.3 Backbone: CSP-сеть	13
2.3.4 Neck: Path Aggregation Network	14
2.3.5 Anchor-free голова и end-to-end инференс без NMS	14
2.3.6 Функция потерь и оптимизация	14
2.3.7 Параметры инференса	15
2.4 Детекция номерных знаков: дообученный YOLO26n	15
2.4.1 Схема transfer learning	15
2.4.2 Параметры обучения	16
2.5 Оптическое распознавание номеров: EasyOCR	16
2.6 Классификация цвета и типа кузова: ResNet18	16
2.6.1 Архитектура ResNet18	16
2.6.2 Двухфазное дообучение	17
2.6.3 Проблема затухающего градиента в глубоких сетях	17
2.6.4 Теоретическое обоснование residual connections	18
2.6.5 ImageNet-предобучение и стратегии transfer learning	18
2.6.6 Гиперпараметры обучения	19
2.6.7 Аугментация данных	19
2.7 Алгоритмы оптимизации	19

2.7.1	Стохастический градиентный спуск с импульсом	19
2.7.2	Adam	20
2.7.3	Планировщики скорости обучения	20
2.8	Методы регуляризации	20
2.8.1	L2-регуляризация (weight decay)	20
2.8.2	Ранняя остановка	21
2.8.3	Аугментация как регуляризация	21
2.8.4	Mixup-аугментация	21
2.8.5	Test-Time Augmentation (TTA)	22
2.9	Сравнимые архитектуры: ResNet50 и EfficientNet-B0	22
2.9.1	ResNet50: bottleneck-архитектура	23
2.9.2	EfficientNet-B0: составное масштабирование	23
2.10	Датасеты	24
2.10.1	VCoR — классификация цвета	24
2.10.2	CompCars — классификация типа кузова	25
3	Описание экспериментов	26
3.1	Эксперимент 1: детекция ТС и номерных знаков	26
3.1.1	Протокол и обоснование гиперпараметров	26
3.1.2	Результаты	27
3.1.3	Сравнение с YOLOv8n	29
3.2	Эксперимент 2: классификатор цвета	30
3.2.1	Протокол и обоснование гиперпараметров	30
3.2.2	Результаты	32
3.3	Эксперимент 3: классификатор типа кузова	35
3.3.1	Протокол и обоснование гиперпараметров	35
3.3.2	Регуляризация: Mixup-аугментация	36
3.3.3	Результаты	38
3.3.4	Подэксперимент: сравнение архитектур ResNet18, ResNet50 и EfficientNet-B0	40
	Постановка и обоснование методологии	40
	Результаты	41
3.4	Эксперимент 4: финальный конвейер	41
3.4.1	Тест на одиночном изображении	41
3.4.2	Видеоинференс с трекингом и покадровой агрегацией характеристик	43
3.4.3	Метрики производительности	44
3.4.4	Итоговая таблица результатов	45
3.4.5	Детальный анализ ошибок OCR	45
3.4.6	Анализ узких мест и практические рекомендации	46
	Заключение	47
	Список использованных источников	48

Обозначения и сокращения

ANPR	Automatic Number Plate Recognition — автоматическое распознавание номерных знаков
BN	Batch Normalization — пакетная нормализация
CNN	Convolutional Neural Network — сверточная нейронная сеть
CRR	Character Recognition Rate — доля верно распознанных символов
DCNN	Deep CNN — глубокая сверточная нейронная сеть
FPN	Feature Pyramid Network — сеть пирамиды признаков
GAP	Global Average Pooling
IoU	Intersection over Union — пересечение по объединению
LR	Learning Rate — скорость обучения
mAP	mean Average Precision — средняя точность по классам
OCR	Optical Character Recognition — оптическое распознавание символов
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent — стохастический градиентный спуск
SiLU	Sigmoid Linear Unit (Swish)
ТС	Транспортное средство
VMMR	Vehicle Make and Model Recognition — распознавание марки и модели ТС
YOLO	You Only Look Once — одноэтапный детектор объектов

Введение

Актуальность задачи

В настоящее время анализ видеоизображений широко применяется для мониторинга дорожного движения, обеспечения безопасности и контроля доступа на объекты. Объём видеоданных постоянно растёт, что делает ручную обработку информации неэффективной и трудоёмкой.

Практический интерес представляет автоматическое определение характеристик автомобиля: государственного регистрационного знака, марки, модели и цвета. Эти данные применяются в задачах контроля проезда, поиска транспортных средств и учёта трафика. Однако качество видеозаписей часто ограничено низким разрешением, изменениями освещённости, погодными условиями и движением объектов.

Цель и задачи работы

Объект исследования — видеоизображения транспортных средств, полученные в реальных условиях эксплуатации.

Предмет исследования — методы и алгоритмы автоматического определения характеристик транспортных средств (номерной знак, цвет кузова, тип кузова) на основе глубокого обучения.

Цель — разработка системы автоматического определения цвета, типа кузова и номерного знака автомобиля по видеопотоку.

Задачи:

1. Провести анализ существующих методов детекции ТС, ANPR, VMMR и определения цвета.
2. Выбрать и обосновать архитектуры моделей для каждой подзадачи.
3. Подготовить обучающие датасеты.
4. Обучить и оценить модели детекции номеров, классификаторы цвета и типа кузова.
5. Интегрировать модели в единый конвейер обработки видео.
6. Провести сравнительный эксперимент ResNet18, ResNet50 и EfficientNet-B0.

Методы исследования: анализ научно-технической литературы; методы глубокого обучения (сверточные нейронные сети, transfer learning); экспериментальный метод — сравнительное тестирование архитектур ResNet18, ResNet50 и EfficientNet-B0; метрический анализ качества моделей (mAP, Accuracy, CRR, IoU).

Научно-теоретическая значимость работы состоит в систематизации и сравнительном анализе современных методов детекции транспортных средств, автоматического распознавания номерных знаков, классификации типа кузова и цвета; в обосновании эффективности конвейерной архитектуры на основе transfer learning для задач видеоаналитики транспорта.

Практическая значимость работы состоит в разработке работоспособного прототипа системы для автоматизации анализа транспортных потоков по видеоизображениям.

Апробация результатов. Результаты работы не выносились на публикацию или конференции; они представлены в форме выпускной квалификационной работы.

Постановка задачи

Задача анализа транспортных средств по видеоданным представляется как последовательность следующих этапов:

1. детекция автомобиля в кадре;

2. локализация номерного знака внутри области автомобиля;
3. оптическое распознавание символов номера;
4. классификация визуальных атрибутов: цвет и тип кузова.

Формально результат детекции объектов на изображении:

$$D = \{(b_i, c_i, p_i)\}_{i=1}^N,$$

где $b_i = (x_i, y_i, w_i, h_i)$ — ограничивающий прямоугольник, c_i — класс объекта, p_i — уверенность детекции.

Качество локализации оценивается через Intersection over Union:

$$\text{IoU} = \frac{S(B_{\text{pred}} \cap B_{\text{gt}})}{S(B_{\text{pred}} \cup B_{\text{gt}})}.$$

1 Обзор существующих методов

1.1 Детекция транспортных средств

1.1.1 Сравнение подходов к детекции

Эволюция методов детекции объектов прошла несколько этапов. Таблица 1.1 суммирует основные подходы.

Таблица 1.1 – Сравнение методов детекции объектов

Метод	Принцип	Достоинства	mAP	FPS
HOG+SVM	Ручные признаки, скользящее окно	Прост в реализации, нет GPU	низкий	5–15
Наар-каскады	Каскад слабых классификаторов	Очень быстрые на CPU	низкий	30+
Faster R-CNN	Двухэтапная сеть, RPN+cls	Высокая точность	высокий	5–15
SSD	Одноэтапная, anchor-based	Баланс скорости и точности	средний	46+
YOLO26n	Одноэтапная, anchor-free, NMS-free	Скорость + точность	высокий	100+

1.1.2 Двухэтапные детекторы (Faster R-CNN)

Детекторы семейства Faster R-CNN работают в два этапа: Region Proposal Network генерирует кандидаты, затем классификационная голова уточняет координаты. Подход обеспечивает высокую точность, но требователен к вычислительным ресурсам и не обеспечивает скорость, достаточную для обработки видео в реальном времени.

1.1.3 Одноэтапные детекторы (YOLO)

Семейство YOLO [1] решает задачу детекции за один проход по сети. Anchor-free архитектура, CSP-backbone и PAN-FPN неск обеспечивают работу на GPU со скоростью 100+ FPS при конкурентоспособной точности. В качестве детектора в работе используется YOLO26n [2] — актуальная модель семейства (2026), которая дополнительно реализует end-to-end инференс без NMS и отказ от Distribution Focal Loss, что упрощает экспорт и снижает задержку (см. раздел 2.3).

Обоснование выбора: YOLO26n оптимален для видеопотока в реальном времени, совместим с форматами ONNX/TensorRT и при меньшем числе параметров превосходит предыдущее поколение по точности на целевых данных (сравнение с YOLOv8n — подраздел 3.1.3).

1.2 Распознавание номерных знаков (ANPR)

Типовая ANPR-система включает несколько последовательных этапов [3, 4], схема которых представлена на рисунке 1.1.

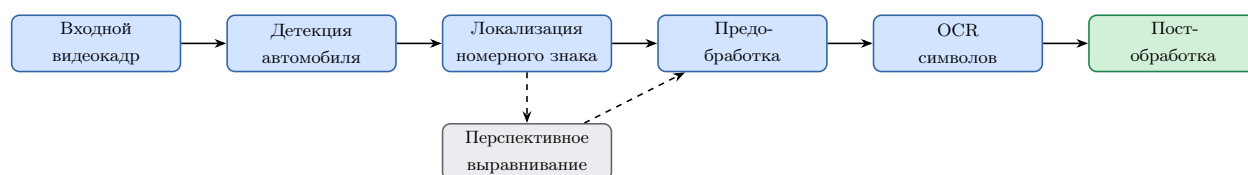


Рисунок 1.1 – Типовая схема ANPR-системы

1.2.1 Сравнение ANPR-подходов

Таблица 1.2 – Сравнение подходов к ANPR

Подход	Описание	mAP plate	Усл. освещения
Морфология + шаблоны	Пороговая обработка + шаблонный OCR	< 0.70	Плохо
HOG + SVM + Tesseract	HOG-дескриптор для локализации + Tesseract OCR	0.75–0.80	Удовлетворительно
YOLO + CRNN	YOLO-детекция + seq2seq OCR	0.88–0.93	Хорошо
YOLO26 + EasyOCR	Anchor-free, NMS-free детекция + CTC OCR	>0.90	Хорошо

Обоснование выбора: YOLO26n (fine-tuned) + EasyOCR обеспечивает готовую поддержку кириллицы без обучения собственной OCR-модели.

1.3 Распознавание марки и типа кузова (VMMR)

Распознавание марки/модели автомобиля (VMMR) относится к задачам тонкой многоклассовой классификации [5, 6, 7]. Задача:

$$\hat{y} = \arg \max_k P(y = k | x), \quad (1.1)$$

где x — изображение, k — класс марки или типа кузова.

Таблица 1.3 – Сравнение архитектур классификаторов для VMMR

Архитектура	Особенности	Params, М	Top-1 Acc	FPS
AlexNet	Первая глубокая CNN для ImageNet	61	0.74–0.80	высокий
VGG16	Глубокая однородная архитектура	138	0.85–0.89	низкий
ResNet18	Residual connections, лёгкая	11.7	0.88–0.92	высокий
ResNet50	Bottleneck residual, глубже и точнее	25.6	0.90–0.93	средний
EfficientNet-B0	Составной масштаб (глубина/ширина/разрешение)	5.3	0.90–0.94	средний

Обоснование выбора ResNet18: оптимальное соотношение точности и скорости; 11.7М параметров позволяют дообучить модель на ограниченных данных без переобучения.

1.4 Определение цвета автомобиля

Цвет автомобиля является значимым визуальным атрибутом для поиска и идентификации транспортных средств [8]. Таблица 1.4 суммирует основные подходы к его классификации.

Таблица 1.4 – Сравнение методов классификации цвета автомобиля

Метод	Принцип	Ассурасу	Устойчивость к освещению
k-means (RGB)	Кластеризация пикселей	0.70–0.81	Низкая
k-means (HSV)	Кластеризация в HSV-пространстве	0.78–0.85	Средняя
SVM + цветовые гистограммы	Гистограммы + SVM	0.83–0.88	Средняя
CNN (ResNet18)	Сквозное обучение	>0.90	Высокая

Для повышения устойчивости в видеопотоке применяется агрегация предсказаний по нескольким кадрам [9]:

$$\hat{y} = \arg \max_k \sum_{t=1}^T \mathbf{1}[y_t = k]. \quad (1.2)$$

Обоснование выбора: CNN (ResNet18) неявно учится инвариантности к освещению и превосходит кластеризационные методы на 10–15%.

2.1 Архитектура системы

Разработанная система представляет собой последовательный конвейер, обрабатывающий каждый кадр видеопотока. Общая схема приведена на рисунке 2.1.

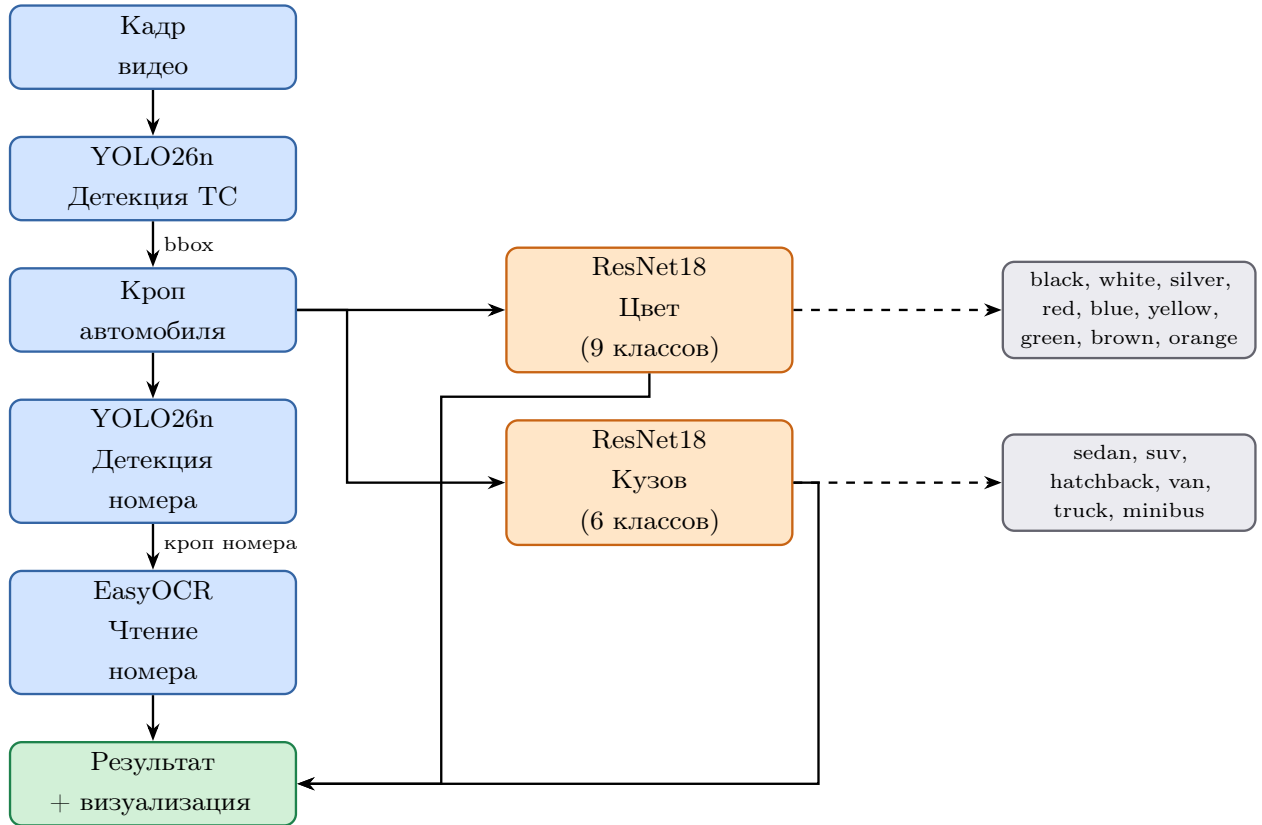


Рисунок 2.1 – Схема конвейера обработки видеокadra

Схема на рисунке 2.1 описывает обработку одиночного кадра. При работе с видеопотоком поверх покадрового конвейера добавляется слой межкадрового сопровождения объектов (трекинг) и агрегации характеристик по каждому транспортному средству; его реализация и обоснование приведены в подразделе 3.4.2.

2.2 Теоретические основы глубокого обучения

Применяемые в работе модели относятся к классу глубоких сверточных нейронных сетей (Deep Convolutional Neural Networks, DCNN). Данный раздел описывает ключевые строительные блоки, необходимые для понимания архитектур YOLO26 и ResNet18.

2.2.1 Сверточные нейронные сети

CNN — специализированный класс нейронных сетей для обработки данных с регулярной сеточной топологией. Три ключевых принципа CNN:

- Разделение весов: одно ядро применяется ко всем позициям входа, что резко сокращает число параметров по сравнению с полносвязным слоем;
- Локальная связность: каждый нейрон реагирует только на рецептивное поле (receptive field) ограниченного размера, что обеспечивает инвариантность к трансляции;

- Иерархия признаков: ранние слои кодируют локальные паттерны (края, текстуры), глубокие — семантические концепции (силуэт кузова, форма номерного знака).

Для одноканального входа $I \in \mathbb{R}^{H \times W}$ и ядра $K \in \mathbb{R}^{m \times n}$ дискретная операция кросс-корреляции (в практике машинного обучения называемая «свёрткой»):

$$(I \star K)[i, j] = \sum_{u=0}^{m-1} \sum_{v=0}^{n-1} I[i+u, j+v] \cdot K[u, v]. \quad (2.1)$$

В многоканальном случае (C_{in} входных каналов, C_{out} выходных) выходная карта признаков:

$$y_k[i, j] = \sum_{c=1}^{C_{\text{in}}} (I_c \star K_{k,c})[i, j] + b_k, \quad k = 1, \dots, C_{\text{out}}, \quad (2.2)$$

где $K_{k,c}$ — ядро k -го фильтра для входного канала c , b_k — смещение (bias).

Число параметров свёрточного слоя: $m \cdot n \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$, что не зависит от пространственного размера входа $H \times W$. Для сравнения, полносвязный слой потребовал бы $H \cdot W \cdot C_{\text{in}}$ параметров на каждый выходной нейрон.

2.2.2 Функции активации

Для введения нелинейности после каждого свёрточного слоя применяется функция активации. В ResNet18 используется ReLU:

$$\text{ReLU}(x) = \max(0, x). \quad (2.3)$$

ReLU не насыщается на положительной полуоси, градиент равен единице для $x > 0$ — это ключевое свойство, предотвращающее затухание градиентов при прямолинейном прохождении.

В YOLO26 применяется SiLU (Sigmoid Linear Unit, также известная как Swish):

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}, \quad (2.4)$$

обеспечивающая плавный градиент вблизи нуля и улучшающая сходимость глубоких детекционных сетей.

2.2.3 Pooling и агрегация признаков

Max-pooling с окном $k \times k$ и шагом s уменьшает пространственное разрешение, сохраняя наиболее значимые активации:

$$y[i, j] = \max_{\substack{u \in [0, k) \\ v \in [0, k)}} x[si + u, sj + v]. \quad (2.5)$$

Global Average Pooling (GAP) применяется в конце классификационных сетей перед FC-слоем:

$$\text{GAP}_c = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W x_c[i, j]. \quad (2.6)$$

GAP сводит пространственную карту к одному числу на канал, что обеспечивает инвариантность к размеру входного изображения и значительно снижает число параметров по сравнению с flatten + FC. В ResNet18 GAP даёт вектор $\mathbb{R}^{512}$ из feature map $7 \times 7 \times 512$.

2.2.4 Пакетная нормализация

Пакетная нормализация (Batch Normalization, BN) нормирует активации по мини-батчу:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta, \quad (2.7)$$

где $\mu_{\mathcal{B}}$ и $\sigma_{\mathcal{B}}^2$ — выборочные среднее и дисперсия по батчу, γ и β — обучаемые параметры масштабирования и сдвига, $\varepsilon \approx 10^{-5}$ — стабилизирующая константа.

Основные эффекты BN:

1. снижение чувствительности к выбору начальной LR;
 2. неявная регуляризация — уменьшает необходимость в Dropout;
 3. ускорение сходимости в 10–14 раз по сравнению с сетями без BN при одинаковом расписании LR.
- BN присутствует в каждом convolution-блоке как ResNet18, так и YOLO26, что обусловило выбор относительно высоких начальных LR в обоих случаях.

2.3 Детекция транспортных средств: YOLO26

2.3.1 Внутренняя архитектура YOLO26

YOLO26 [2] — актуальное поколение одноэтапных детекторов Ultralytics (2026). Как и предшественники, модель состоит из трёх функциональных блоков (рисунок 2.2): backbone для извлечения признаков, neck для их многомасштабного слияния и head для предсказания боксов и классов. Принципиальные отличия YOLO26 от предыдущих поколений сосредоточены не в топологии, а в способе вывода и обучения:

- End-to-end инференс без NMS — модель напрямую выдаёт финальный набор детекций, исключая постобработку Non-Maximum Suppression и связанную с ней задержку;
- отказ от Distribution Focal Loss (DFL) — координаты бокса регрессируются напрямую, что упрощает экспорт в ONNX/TensorRT и ускоряет вывод на CPU;
- ProgLoss (Progressive Loss Balancing) — динамическая балансировка компонентов функции потерь для устойчивого обучения;
- STAL (Small-Target-Aware Label assignment) — назначение меток с учётом мелких объектов, повышающее их полноту (recall);
- оптимизатор MuSGD — гибрид Muon и SGD с более стабильной и быстрой сходимостью.

Эти изменения дают на COCO mAP@0.50:0.95 = 40.9 для версии nano против 37.3 у YOLOv8n при меньшем числе параметров (2.4M против 3.2M) и FLOPs (5.4 против 8.7 GFLOPs).

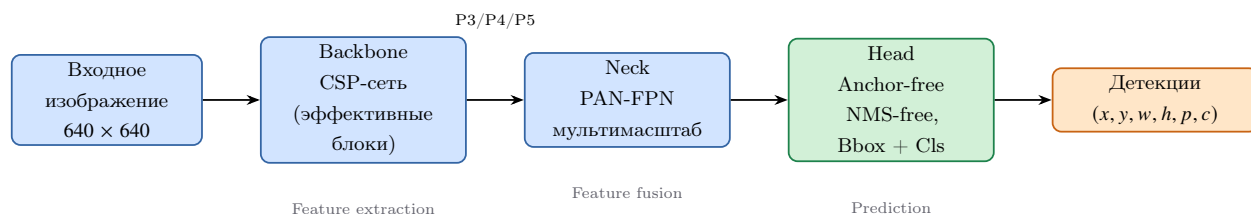


Рисунок 2.2 – Упрощённая схема архитектуры YOLO26

Топология backbone–neck–head унаследована от предыдущих поколений семейства; ниже подробно рассматриваются backbone (подраздел 2.3.3), neck (подраздел 2.3.4), anchor-free голова с end-to-end инференсом без NMS (подраздел 2.3.5), а также функция потерь и оптимизатор (подраздел 2.3.6).

2.3.2 Эволюция семейства YOLO

Семейство YOLO прошло путь от первой версии (2016) до YOLO26 (2026), последовательно устраняя ключевые ограничения предыдущих поколений.

Таблица 2.1 – Эволюция ключевых характеристик архитектур YOLO

Версия	Год	Ключевые нововведения	mAP@0.5
YOLOv1	2016	Сетка $S \times S$; B боксов на ячейку; softmax классов	63.4
YOLOv2	2017	Anchor-boxes, BatchNorm, многомасштабное обучение	78.6
YOLOv3	2018	Darknet-53; детекция на 3 масштабах; sigmoid классификаторы	82.1
YOLOv4	2020	CSPDarknet53, PANet, CIoU loss, Mosaic-аугментация	84.0
YOLOv5	2020	AutoAnchor, Focus-слой, адаптивный batching (PyTorch)	84.7
YOLOv6	2022	RepVGG backbone, знаниевая дистилляция	85.0
YOLOv7	2022	Extended-ELAN, составное масштабирование	85.6
YOLOv8	2023	Anchor-free, C2f блоки, decoupled head, DFL loss	86.3
YOLO26	2026	End-to-end (NMS-free), отказ от DFL, ProgLoss + STAL, MuSGD	—

*Для YOLO26 в столбце mAP приведён прочерк, так как характеристика измеряется иначе: на COCO версия nano даёт mAP@0.5:0.95 = 40.9 против 37.3 у YOLOv8n [2].

Переход к anchor-free подходу произошёл на поколении 2023 года и унаследован YOLO26. В классических anchor-based детекторах (YOLOv2–v5) необходимо предварительно кластеризовать размеры объектов в датасете для выбора наборов якорей. Anchor-free голова вместо этого напрямую предсказывает геометрию бокса, что упрощает адаптацию к нестандартным типам объектов (например, номерные знаки имеют специфическое соотношение сторон $\approx 4:1$, плохо покрываемое универсальными COCO-якорями). В YOLO26 этот принцип получил продолжение: модель отказывается не только от якорей, но и от постобработки NMS, формируя итоговый набор детекций за один проход (подраздел 2.3.5).

2.3.3 Backbone: CSP-сеть

В основе backbone YOLO26 лежит схема Cross-Stage Partial Networks (CSP). Её идея в том, что входной тензор делится на две ветви: первая обрабатывается стеком свёрточных блоков (dense path), вторая передаётся дальше без изменений (identity path); затем обе ветви сшиваются конкатенацией и пропускаются через свёртку 1×1 :

$$\text{CSP}(X) = \text{Conv}_{1 \times 1}(\text{Concat}[\text{DenseBlock}(X_{C/2}), X_{C/2}]), \quad (2.8)$$

где $X_{C/2}$ и $X_{C/2}$ — первая и вторая половины каналов тензора $X \in \mathbb{R}^{C \times H \times W}$. Благодаря такому разделению вычислительная стоимость падает примерно на 30% FLOPS, а identity path при этом удерживает насыщенный градиентный поток.

Многоветвевые CSP-блоки (потомки связки C3→C2f, отлаженной в поколениях 2020–2023 годов) доносят все промежуточные feature maps до финальной конкатенации; за счёт этого растёт число независимых градиентных путей, тогда как параметров прибавляется немного. Отдельно backbone YOLO26 переработан ради экономии FLOPs — nano-версия укладывается в 5.4 GFLOPs против 8.7 у YOLOv8n и при этом точнее на COCO.

2.3.4 Neck: Path Aggregation Network

Объекты в кадре сильно расходятся по размеру (далёкие номерные знаки малы, ближние автомобили крупны), поэтому YOLO26 опирается на связку PAN-FPN (Path Aggregation Network поверх Feature Pyramid Network).

FPN (top-down path): формирует пирамиду признаков и подмешивает семантику глубоких (абстрактных) уровней в более детальные поверхностные:

$$P_k = \text{Conv}(\text{Upsample}(P_{k+1}) + C_k), \quad (2.9)$$

где C_k — выход backbone на уровне k , P_k — обогащённая feature map.

PAN (bottom-up path): достраивает к этому восходящую ветвь; она подтягивает признаки высокого разрешения с нижних уровней и тем самым уточняет пространственную локализацию:

$$N_k = \text{Conv}(\text{Stride-Conv}(N_{k-1}) + P_k). \quad (2.10)$$

При входном разрешении 640×640 предсказание ведётся на трёх головах:

- P3 (80×80) — мелкие объекты (далёкие номерные знаки);
- P4 (40×40) — объекты среднего масштаба;
- P5 (20×20) — крупные объекты (автомобили вблизи).

2.3.5 Anchor-free голова и end-to-end инференс без NMS

Для каждой ячейки YOLO26 предсказывает четыре расстояния от её центра до сторон бокса (l, t, r, b) , по которым затем собирается ограничивающий прямоугольник:

$$x_1 = x_c - l, \quad y_1 = y_c - t, \quad x_2 = x_c + r, \quad y_2 = y_c + b. \quad (2.11)$$

Отказ от Distribution Focal Loss. В версии 2023 года координаты получали через DFL: голова предсказывала дискретное распределение по бинам, а итоговое значение бралось как его математическое ожидание. В YOLO26 этот механизм убран, и координаты регрессируются напрямую. Лишняя ветвь в голове исчезает, граф вычислений упрощается, инференс ускоряется (по оценкам Ultralytics — до 43% на CPU), а заодно снимается давняя проблема экспорта в ONNX/TensorRT, где слой DFL приходилось поддерживать отдельно.

End-to-end инференс без NMS. Обычные детекторы, в том числе YOLOv8, выдают на каждый объект по несколько перекрывающихся боксов, а лишние отсекает Non-Maximum Suppression (NMS) по порогу IoU. Сам по себе NMS — это последовательная, плохо распараллеливаемая постобработка со своей задержкой и дополнительным гиперпараметром. В YOLO26 метки на обучении раздаются по схеме «один-к-одному», поэтому голова сразу отдаёт непересекающийся набор детекций и NMS не требуется. Сквозная задержка снижается, а время инференса перестаёт зависеть от числа кандидатов в кадре, то есть становится детерминированным.

2.3.6 Функция потерь и оптимизация

Без DFL итоговая функция потерь YOLO26 распадается на две составляющие — локализационную и классификационную:

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}. \quad (2.12)$$

$\mathcal{L}_{\text{box}}$ — Complete IoU (CIoU) loss одновременно отслеживает три геометрических признака совпадения боксов:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{\text{gt}})}{c^2} + \alpha v, \quad (2.13)$$

где $\rho^2(b, b^{\text{gt}})$ — квадрат евклидова расстояния между центрами предсказанного и эталонного боксов;
 c — диагональ минимального объемлющего прямоугольника;

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w^{\text{gt}}}{h^{\text{gt}}} - \arctan \frac{w}{h} \right)^2 \quad (2.14)$$

— штраф за расхождение соотношений сторон; $\alpha = v / (1 - \text{IoU} + v)$ — нормирующий коэффициент. Главное отличие от обычного IoU-loss в том, что CIOU не обнуляет градиент даже когда боксы не пересекаются, и за счёт этого сходится быстрее.

$\mathcal{L}_{\text{cls}}$ — Binary Cross-Entropy (BCE) считается отдельно для каждого класса поверх сигмоидной активации:

$$\mathcal{L}_{\text{cls}} = - \sum_{k=1}^K [y_k \log \hat{p}_k + (1 - y_k) \log(1 - \hat{p}_k)], \quad (2.15)$$

где $\hat{p}_k = \sigma(z_k)$ — предсказанная вероятность класса k .

Сверх этого в обучение YOLO26 добавлены два приёма:

- ProgLoss (Progressive Loss Balancing) по ходу обучения постепенно перераспределяет веса между $\mathcal{L}_{\text{box}}$ и $\mathcal{L}_{\text{cls}}$, благодаря чему ранние эпохи проходят устойчивее;
- STAL (Small-Target-Aware Label assignment) при разметке положительных меток учитывает мелкие объекты и поднимает их recall — что напрямую важно для удалённых номерных знаков.

Оптимизатор MuSGD сочетает свойства Muon и SGD, обеспечивая более стабильную и быструю сходимость, чем чистый SGD; в Ultralytics он используется как оптимизатор по умолчанию для YOLO26.

2.3.7 Параметры инференса

Таблица 2.2 – Параметры запуска YOLO26n для детекции ТС

Параметр	Значение
Модель	yolo26n.pt (предобученная, COCO)
Порог уверенности	$p_{\min} = 0.40$
Постобработка	end-to-end, без NMS
Используемые классы	2 (car), 3 (motorcycle), 5 (bus), 7 (truck)
Размер входа	640×640

2.4 Детекция номерных знаков: дообученный YOLO26n

2.4.1 Схема transfer learning

Процесс дообучения показан на рисунке 2.3.

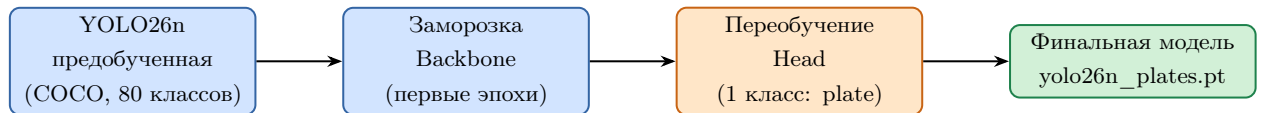


Рисунок 2.3 – Схема transfer learning для детектора номерных знаков

2.4.2 Параметры обучения

Таблица 2.3 – Гиперпараметры дообучения детектора номеров

Параметр	Значение
Базовая модель	yolo26n.pt
Эпохи	50
Размер входа	640×640
Размер батча	16
Оптимизатор	optimizer=auto (Ultralytics)
Начальная LR	подбирается автоматически
Weight decay	5×10^{-4}
Число классов	1 (license_plate)
Метрика сохранения	mAP@0.50 на тестовой выборке

2.5 Оптическое распознавание номеров: EasyOCR

EasyOCR использует двухэтапный конвейер, представленный на рисунке 2.4.

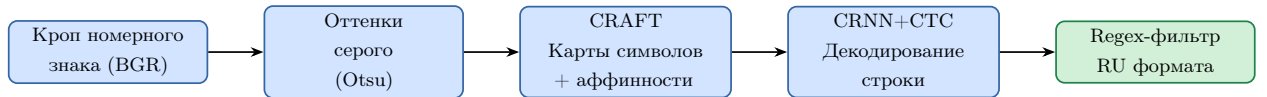


Рисунок 2.4 – Конвейер EasyOCR для распознавания номерного знака

Качество OCR оценивается через Character Error Rate:

$$\text{CER} = \frac{S + D + I}{N}, \quad (2.16)$$

где S, D, I — замены, удаления, вставки (алгоритм Левенштейна), N — длина эталонной строки.

2.6 Классификация цвета и типа кузова: ResNet18

2.6.1 Архитектура ResNet18

Ключевая идея ResNet — shortcut-соединения:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}, \quad (2.17)$$

что устраняет затухание градиентов и позволяет строить глубокие сети. Схема residual-блока и размещение слоёв в ResNet18 приведены на рисунке 2.5.

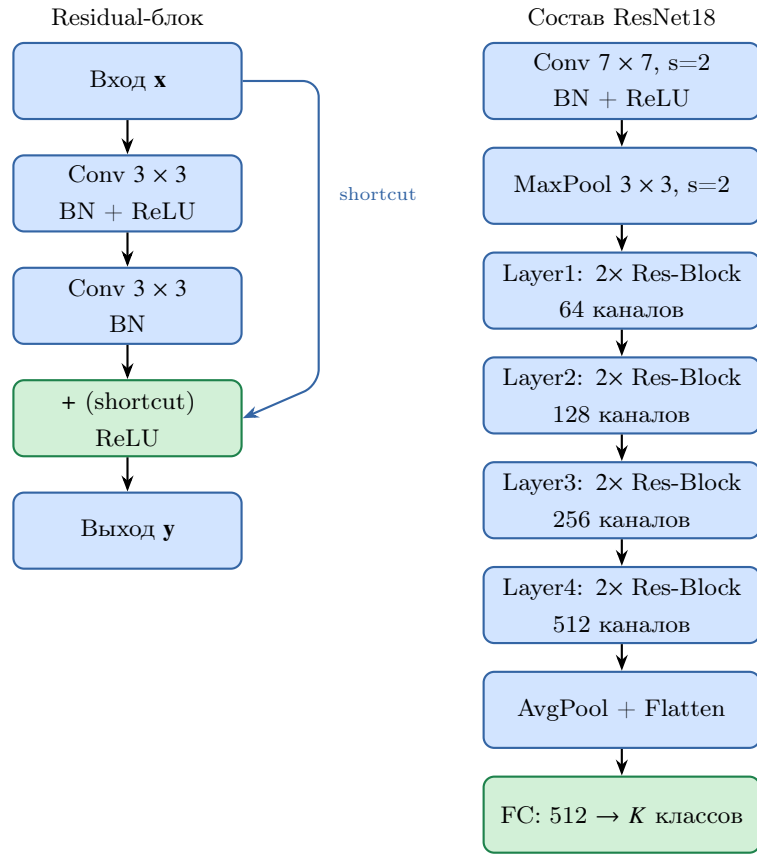


Рисунок 2.5 – Residual-блок и структура ResNet18 (18 слоёв, K — число классов)

2.6.2 Двухфазное дообучение

Стратегия двухфазного обучения показана на рисунке 2.6.

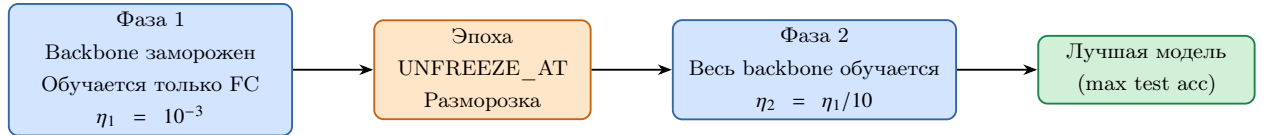


Рисунок 2.6 – Стратегия двухфазного дообучения ResNet18

Функция потерь — кросс-энтропия:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^K y_k \log \hat{y}_k, \quad (2.18)$$

где y_k — one-hot метка, $\hat{y}_k$ — предсказанная вероятность.

2.6.3 Проблема затухающего градиента в глубоких сетях

В алгоритме обратного распространения (backpropagation) производная функции потерь по активациям слоя l раскрывается по цепному правилу:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \prod_{k=l}^{L-1} \frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k}, \quad (2.19)$$

где $\mathbf{x}_k$ — активации слоя k , L — глубина сети.

Когда $\|J_k\| < 1$, произведение якобианов $\prod_{k=l}^{L-1} J_k$ убывает экспоненциально с глубиной: до нижних слоёв доходит почти нулевой градиент, и они перестают обучаться.

2.6.4 Теоретическое обоснование residual connections

He et al. (2015) исходили из простого наблюдения: вместо прямого отображения $\mathcal{H}(\mathbf{x})$ сети проще выучить его остаток $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (2.20)$$

Если искомое отображение почти тождественно, модели достаточно увести $\mathcal{F}$ к нулю, а для стека нелинейных слоёв это несложная задача.

Анализ градиентного потока. С учётом shortcut-соединений производная потерь записывается так:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \underbrace{\prod_{k=l}^{L-1} \left(1 + \frac{\partial \mathcal{F}_k}{\partial \mathbf{x}_k}\right)}_{\text{не обращается в нуль}}. \quad (2.21)$$

Слагаемое-единица в каждом сомножителе не даёт всему произведению обратиться в нуль независимо от глубины. Это и есть теоретическое объяснение того, почему ResNet удаётся обучать в диапазоне от 18 до 1000 слоёв.

Почему ResNet18, а не более глубокие версии. На небольших датасетах (<50k изображений) компактные сети нередко обходят ResNet50/101 просто потому, что меньше переобучаются. Для VCoR ($\approx 15k$ изображений) и CompCars ($\approx 30k$) ResNet18 как раз и даёт удачный баланс между ёмкостью модели и риском переобучения.

2.6.5 ImageNet-предобучение и стратегии transfer learning

Веса, предобученные на ImageNet (1.28M изображений, 1000 классов), несут в себе универсальные визуальные признаки, которые удобно разнести по трём уровням:

- layer1–layer2: края, ориентации, текстуры;
- layer3: фрагменты объектов — округлые формы, угловые элементы;
- layer4: семантика высокого уровня, в том числе специфика автомобилей.

В работе задействованы три сценария transfer learning:

1. Feature extraction (полная заморозка backbone): дообучается только FC-голова. Вариант быстрый, но негибкий — готовые признаки не всегда подходят целевым данным.
2. Partial fine-tuning: размораживаются верхние слои backbone (layer3+layer4+FC); именно этот режим применяется в классификаторе кузова.
3. Full fine-tuning: обучается весь backbone, но с LR в десять раз меньше, чем у FC. Так дообучался классификатор цвета (фаза 2); режим требует достаточного объёма целевых данных.

Обоснование двухфазной стратегии (фаза 1: заморозка, фаза 2: разморозка): в начале дообучения FC-веса случайны и порождают большие градиенты. Если одновременно обучать backbone, эти большие градиенты разрушат предобученные признаки (catastrophic forgetting). Заморозка backbone на первых эпохах стабилизирует FC, после чего backbone размораживается с малой LR — это позволяет тонко адаптировать признаки к целевым данным (изображения автомобилей в реальных условиях съёмки).

2.6.6 Гиперпараметры обучения

Таблица 2.4 – Гиперпараметры обучения классификаторов ResNet18

Параметр	Классификатор цвета	Классификатор кузова
Число классов	9	6
Макс. эпох	20	40 (early stop)
Разморозка	эп. 5 (весь backbone)	с эп. 6 (layer3+layer4+fc)
Батч	32	32
LR	$10^{-3} \rightarrow 10^{-4}$	5×10^{-5}
Оптимизатор	Adam	Adam
β_1, β_2	0.9, 0.999	0.9, 0.999
StepLR шаг	5 эпох	—
StepLR γ	0.3	—
Early stopping	—	patience = 7
Размер входа	224×224	224×224

2.6.7 Аугментация данных

Таблица 2.5 – Аугментации для обучения классификаторов

Преобразование	Параметры	Назначение
Обучающая выборка		
RandomResizedCrop	224, scale [0.7, 1.0]	Масштаб и позиция
RandomHorizontalFlip	$p = 0.5$	Зеркальное отражение
ColorJitter	brightness=0.3, contrast=0.3	Освещение
RandomRotation	$\pm 10^\circ$	Поворот
Normalize	$\mu = [.485, .456, .406]$	ImageNet stats
Валидационная выборка		
Resize	224×224	Размер входа
Normalize	$\mu = [.485, .456, .406]$	ImageNet stats

2.7 Алгоритмы оптимизации

2.7.1 Стохастический градиентный спуск с импульсом

Базовый SGD обновляет параметры по градиенту на мини-батче $\mathcal{B}_t$:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t; \mathcal{B}_t), \quad (2.22)$$

где η — скорость обучения (learning rate).

SGD с импульсом (momentum) накапливает «инерцию» в направлении постоянного градиента, ускоряя сходимость:

$$v_{t+1} = \mu v_t + \nabla_{\theta} \mathcal{L}(\theta_t; \mathcal{B}_t), \quad \theta_{t+1} = \theta_t - \eta v_{t+1}, \quad (2.23)$$

где μ — коэффициент импульса (типичное значение для детекторов семейства YOLO — $\mu = 0.937$).

Выбор оптимизатора для детектора YOLO26: Детектор номеров обучался с флагом optimizer=auto: на датасетах менее 10 000 изображений Ultralytics сама выбирает и алгоритм оптимизации, и стартовую скорость обучения. Базовым алгоритмом для YOLO26 при этом служит MuSGD — связка Muon и SGD, которая сходится стабильнее и быстрее классического SGD с импульсом. При небольшом

целевом наборе и коротком расписании такой автоподбор снимает необходимость вручную подбирать LR и всё равно даёт устойчивую сходимость.

2.7.2 Adam

Adam (Adaptive Moment Estimation) задаёт каждому параметру свою скорость обучения, отслеживая первый и второй моменты градиента:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (2.24)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (2.25)$$

где $g_t = \nabla_{\theta} \mathcal{L}_t$. После коррекции смещения (debiasing):

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (2.26)$$

Обновление параметров:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t, \quad (2.27)$$

где $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

Почему Adam для классификаторов. Adam укладывается в 10–30 эпох и слабо зависит от выбора стартовой LR. При fine-tuning это особенно удобно: адаптивный шаг не даёт чрезмерно пересчитывать слои, в которых уже хранятся полезные признаки ImageNet.

2.7.3 Планировщики скорости обучения

StepLR каждые S эпох уменьшает LR в γ раз:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/S \rfloor}. \quad (2.28)$$

Для классификатора цвета взяты $S = 5$, $\gamma = 0.3$; LR при этом проходит цепочку $\eta_0 = 10^{-3} \rightarrow 3.4 \times 10^{-4} \rightarrow 10^{-4} \rightarrow$ и т.д.

Логика простая: крупный шаг в начале даёт быстрый прогресс, а уменьшающийся ближе к концу — аккуратную доводку у оптимума.

Линейное затухание LR использовалось при обучении детектора (по умолчанию `cos_lr=False`, `lrf=0.01`):

$$\eta_t = \eta_0 \left(1 - (1 - \text{lrf}) \frac{t}{T} \right), \quad (2.29)$$

здесь LR линейно идёт от η_0 к $\eta_0 \cdot \text{lrf}$ к концу обучения. Такое плавное снижение помогает детектору осесть в устойчивый минимум потерь.

2.8 Методы регуляризации

Под регуляризацией понимают набор приёмов, которые сдерживают переобучение и улучшают обобщающую способность модели.

2.8.1 L2-регуляризация (weight decay)

К функции потерь добавляется штраф на ℓ_2 -норму весов:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \|\theta\|_2^2. \quad (2.30)$$

В данной работе $\lambda = 5 \times 10^{-4}$ для детектора номеров. Weight decay ограничивает «свободу» модели и эквивалентен MAP-оценке с гауссовым априором на веса $\theta \sim \mathcal{N}(0, 1/\lambda)$.

2.8.2 Ранняя остановка

Если контрольная метрика не улучшается в течение patience эпох, обучение прекращается, а лучшая (по тестовой метрике) модель сохраняется. Условие остановки обучения:

$$e - e_{\text{best}} > \text{patience}, \quad (2.31)$$

где e_{best} — номер эпохи с наилучшей метрикой.

В данной работе early stopping с patience=7 настроен для классификатора кузова: обучение прошло все 40 эпох (ранняя остановка не сработала), а лучшая по валидации модель зафиксирована на эпохе 34.

2.8.3 Аугментация как регуляризация

Аугментация увеличивает эффективный объём обучающей выборки без сбора новых данных. Математически оптимизируется:

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \mathbb{E}_{t \sim \mathcal{T}} \mathcal{L}(f_{\theta}(t(x)), y), \quad (2.32)$$

где $\mathcal{T}$ — семейство реалистичных трансформаций.

Аугментации в таблице 2.5 подобраны с учётом реалистичности для видеопотока:

- RandomHorizontalFlip: автомобили въезжают с обеих сторон кадра — зеркальные копии равновероятны;
- ColorJitter (brightness=0.3, contrast=0.3): освещение меняется в течение суток и при смене погоды;
- RandomRotation $\pm 10^\circ$: небольшой наклон возможен при установке камеры под углом;
- RandomResizedCrop (scale [0.7, 1.0]): имитирует различное расстояние до автомобиля; нижняя граница 0.7 выбрана так, чтобы сохранить достаточно цветовой информации кузова.

Состав аугментаций для двух задач неодинаков. Классификатор цвета обучался на базовом наборе (таблица 2.5), где намеренно оставлены цветовые искажения (ColorJitter) — без них теряется устойчивость к освещению. Для типа кузова тот же набор пришлось усилить под несоответствие условий съёмки CompCars $\rightarrow$ улица. Добавлены RandomPerspective (distortion 0.3, $p = 0.5$), воспроизводящий уличные ракурсы 3/4; GaussianBlur ($p = 0.25$) под низкое разрешение и смаз дальних кадров; RandomErasing ($p = 0.3$) под частичные перекрытия. ColorJitter здесь тоже сохранён, но с насыщенностью, сниженной до 0.15; нижняя граница RandomResizedCrop поднята до 0.72, а перед FC-слоем для дополнительной регуляризации поставлены Dropout 0.5 и сглаживание меток (label smoothing 0.1).

2.8.4 Mixup-аугментация

Привычные аугментации — отражения, кропы, цветовые сдвиги — меняют каждое изображение по отдельности и его метку не трогают. Mixup [10] устроен иначе: он берёт два случайных снимка из обучающей выборки и «подмешивает» один к другому, складывая вместе с картинками и их метки. Получается новый, синтетический пример — что-то среднее между двумя исходными.

Зачем это нужно. Обычно модель видит только сами обучающие примеры и ничего не знает о том, что находится «между» ними. Из-за этого она часто слишком уверена в себе и плохо ведёт себя на промежуточных, непохожих на обучающие случаях. Mixup как раз заполняет эти промежутки: показывая модели смеси примеров, он заставляет её вести себя предсказуемо не только в самих обучающих точках, но и в их окрестности (формально это называют минимизацией вицинального риска, VRM).

Как строится смешанный пример. Для двух примеров (x_i, y_i) и (x_j, y_j) изображение и метку смешивают с одним и тем же весом λ :

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha), \quad (2.33)$$

где $\lambda \in [0, 1]$ задаёт пропорцию, в которой берутся два примера. Метка $\tilde{y}$ при этом перестаёт быть «жёсткой»: вместо одного класса она указывает на смесь двух (например, «на 70% седан и на 30% хэтчбек»). Поэтому и ошибку считают как взвешенную сумму ошибок по обоим исходным классам:

$$\mathcal{L}(\tilde{x}, \tilde{y}) = \lambda \text{CE}(f(\tilde{x}), y_i) + (1 - \lambda) \text{CE}(f(\tilde{x}), y_j). \quad (2.34)$$

За что отвечает параметр α . Долю смешивания λ выбирают случайно из распределения $\text{Beta}(\alpha, \alpha)$, и параметр α управляет тем, насколько сильно перемешиваются примеры:

- при маленьком α (например, 0.2) значение λ почти всегда близко к 0 или 1 — смеси «лёгкие», в каждой явно преобладает один из двух снимков;
- при $\alpha = 1$ любая пропорция равновероятна;
- при большом α чаще получаются смеси «пополам» ($\lambda \approx 0.5$) — примеры перемешаны сильнее всего.

На практике берут $\alpha \in [0.1, 0.4]$: смешивание уже даёт нужный эффект, но снимки остаются достаточно «читаемыми», чтобы обучающий сигнал не размывался.

Почему Миксир снижает переобучение. Складывается из нескольких причин:

1. мягкие метки не дают модели быть уверенной на 100% и делают её устойчивее к шумным примерам;
2. обучаясь на «средних» между классами картинках, модель проводит более плавные границы между классами, а такие границы лучше переносятся на новые данные;
3. смешанные примеры лежат как раз на стыке классов — то есть модель заранее тренируется на самых трудных, пограничных случаях.

Особенность метрики на обучении. При Миксир точность на обучающей выборке выглядит заниженной: модель учат на смешанных картинках с мягкими метками, а точность считают по одному «жёсткому» классу. Поэтому train accuracy здесь оказывается ниже валидационной — это нормально и не означает недообучения.

2.8.5 Test-Time Augmentation (TTA)

В отличие от обучающих аугментаций и Миксир, действующих на этапе обучения, Test-Time Augmentation повышает точность на этапе инференса без дообучения модели. Изображение пропускается через сеть в нескольких аугментированных вариантах, а итоговое распределение вероятностей усредняется. Для классификатора типа кузова применяется TTA по горизонтальному отражению:

$$\hat{p}(x) = \frac{1}{2} \left(\text{softmax}(f(x)) + \text{softmax}(f(\text{flip}(x))) \right), \quad (2.35)$$

где $\text{flip}(x)$ — зеркально отражённое по горизонтали изображение. Усреднение по двум видам снижает дисперсию предсказаний и устраняет случайные ошибки, чувствительные к ориентации объекта. Приём вычислительно дешёв (две прямых прогонки вместо одной) и не требует изменения весов; в эксперименте он дал прирост Accuracy top-1 около 0.5–1.0 п.п.

2.9 Сравнимые архитектуры: ResNet50 и EfficientNet-B0

Для объективной оценки качества предобученного backbone классификатор ResNet18 сравнивается с двумя альтернативами: более глубокой ResNet50 (bottleneck-блоки, 23.5 М параметров) и EfficientNet-B0 (составное масштабирование, 4.0 М параметров). Базовым строительным блоком

EfficientNet-B0 является MBConv — инвертированный residual-блок с depthwise-свёрткой и squeeze-excitation (рисунок 2.7): входные карты признаков сначала расширяются свёрткой 1×1 , обрабатываются depthwise-свёрткой 3×3 и проецируются обратно свёрткой 1×1 со skip-соединением.

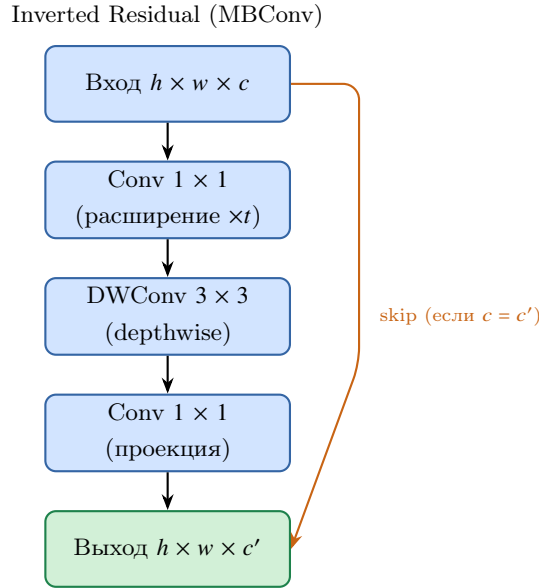


Рисунок 2.7 – Инвертированный residual-блок (MBConv) — основной блок EfficientNet-B0

Таблица 2.6 – Сравнимые архитектуры классификатора типа кузова

Характеристика	ResNet18	ResNet50	EfficientNet-B0
Params, M	11.2	23.5	4.0
Blocks	Residual	Bottleneck	MBConv (inv. res. + SE)
Conv type	Standard	Standard	Depthwise sep.
FLOPS (224)	1.8G	4.1G	0.39G

2.9.1 ResNet50: bottleneck-архитектура

ResNet50 [11] углубляет идею residual-обучения за счёт bottleneck-блоков. В отличие от базового блока ResNet18 из двух свёрток 3×3 , bottleneck-блок применяет последовательность $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$: первая свёртка 1×1 снижает число каналов (сжатие), свёртка 3×3 обрабатывает сжатое представление, а финальная 1×1 восстанавливает размерность. Такая схема позволяет нарастить глубину до 50 слоёв при умеренной вычислительной стоимости (4.1G FLOPS, 25.6 М параметров в исходной ImageNet-версии). Большая глубина формирует более богатую иерархию признаков, что в проведённом эксперименте (подраздел 3.3.4) выразилось в наивысшей точности в режиме feature extraction (0.652). Платой выступает практически двукратное увеличение задержки инференса по сравнению с ResNet18.

2.9.2 EfficientNet-B0: составное масштабирование

EfficientNet [12] основан на принципе составного масштабирования (compound scaling): глубина d , ширина w и разрешение входа r масштабируются согласованно через единый коэффициент ϕ :

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi, \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \quad (2.36)$$

где константы α, β, γ найдены поиском по сетке на базовой модели. EfficientNet-B0 построена из блоков MBConv (рисунок 2.7) с модулем squeeze-excitation (SE) [13], который адаптивно перевзвешивает каналы признаков:

$$s_c = \sigma(W_2 \delta(W_1 z))_c, \quad z_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W u_c(i, j), \quad (2.37)$$

где z_c — глобальный average pooling по каналу c , δ — ReLU, σ — сигмоида, а выход $s_c \in [0, 1]$ масштабирует исходную карту признаков u_c . Благодаря depthwise-свёрткам и SE-механизму EfficientNet-B0 достигает конкурентной точности при наименьшем числе параметров (4.0 М, 0.39G FLOPS). Однако depthwise-операции обладают низкой арифметической интенсивностью и плохо утилизируют параллельные блоки GPU, поэтому на T4 модель показала наименьший FPS среди трёх архитектур (подраздел 3.3.4).

2.10 Датасеты

Таблица 2.7 — Сводная информация об используемых датасетах

Датасет	Задача	Источник	Изобр.	Классов	Формат
COCO 2017	Детекция ТС	COCO/Ultralytics	118k	80 (4 для ТС)	YOLO
Russian license plates	Детекция номеров	Kaggle (tomilovivan)	668	1	YOLO txt
NOMEROFF-NET RU	Оценка OCR	nomeroff.net.ua	2845	—	JSON ann.
VCoR	Цвет авто	Kaggle	$\approx 15k$	9	ImageFolder
CompCars body-type	Тип кузова	CUNHK	$\approx 30k$	6	ImageFolder

2.10.1 VCoR — классификация цвета

Таблица 2.8 — Классы датасета VCoR и соответствующие цвета

#	Класс	Приблизительный RGB	Hex
1	black	(25, 25, 25)	#191919
2	white	(230, 230, 230)	#E6E6E6
3	silver	(175, 180, 185)	#AFB4B9
4	red	(195, 30, 30)	#C31E1E
5	blue	(30, 60, 200)	#1E3CC8
6	yellow	(220, 200, 20)	#DCC814
7	green	(30, 145, 30)	#1E911E
8	brown	(100, 60, 30)	#643C1E
9	orange	(215, 105, 20)	#D7691E

2.10.2 CompCars — классификация типа кузова

Таблица 2.9 – Машинг оригинальных классов CompCars на целевые

Оригинальный класс CompCars	Целевой класс	Описание
sedan, fastback	sedan	Легковой, 4 двери
hatchback, estate	hatchback	Хетчбэк, универсал
SUV, crossover	suv	Внедорожник
MPV	van	Минивэн
pickup, truck	truck	Грузовой / пикап
microbus, minibus	minibus	Микроавтобус

Из класса sedan исключены исходные типы sports и convertible: визуально они существенно отличаются от обычного седана (низкий двухдверный силуэт, открытый верх) и размывали класс, понижая его precision. После очистки класс sedan стал однороднее, что повысило качество соседнего hatchback.

3 Описание экспериментов

3.1 Эксперимент 1: детекция ТС и номерных знаков

Детекция в конвейере двухступенчатая: транспортные средства находит предобученная на COCO YOLO26n (классы car, bus, truck, motorcycle), а внутри области каждого ТС номерной знак локализует та же YOLO26n, дообученная под единственный класс license_plate. Пример работы детектора ТС на реальном кадре приведён на рисунке 3.1.



Рисунок 3.1 – Детекция транспортных средств предобученной на COCO YOLO26n: ограничивающие рамки с оценкой уверенности

3.1.1 Протокол и обоснование гиперпараметров

YOLO26n дообучался на датасете Russian license plates (Kaggle, tomilovivan, 668 изображений: 542 train / 126 test) в течение 50 эпох (параметры — таблица 2.3). Характерные изображения обучающей выборки с разметкой приведены на рисунке 3.2.

Датасет номерных знаков — train (542 изображений)
Зелёный прямоугольник = YOLO-аннотация

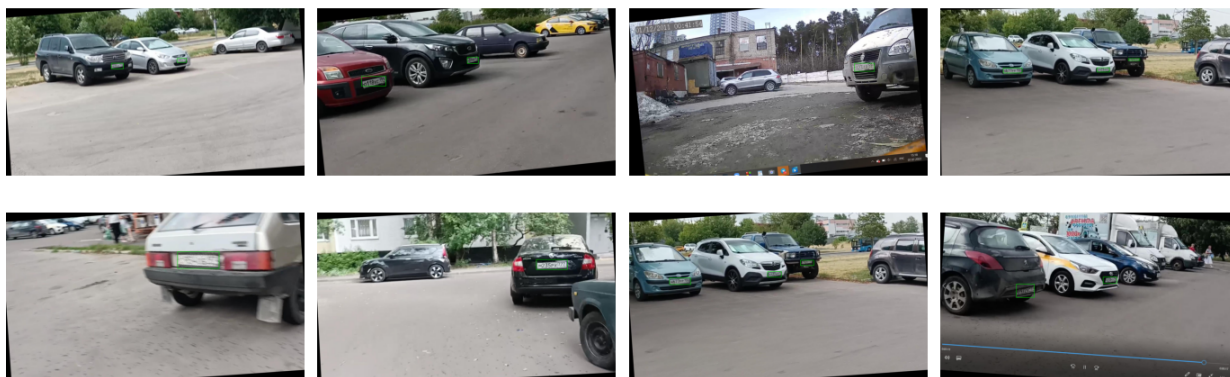


Рисунок 3.2 – Примеры обучающей выборки Russian license plates (542 изображения): зелёным выделена YOLO-разметка номерного знака

Обоснование ключевых решений:

- Transfer learning, а не обучение с нуля: 542 обучающих изображения — слишком мало, чтобы обучить детектор со случайной инициализацией: сеть переобучилась бы, не успев выучить устойчивые признаки. Веса yolo26n.pt, предобученные на COCO (118k изображений, 80 классов), уже кодируют универсальные визуальные признаки (края, формы, фрагменты объектов), поэтому дообучение под единственный класс «номерной знак» сводится к переспециализации головы и аккуратной подстройке backbone. Это обеспечивает быструю сходимость на малом целевом наборе; эффективность подхода подтверждается приростом mAP@0.50 с ≈ 0.503 до 0.9669 после дообучения (таблица 3.1).
- Базовая модель yolo26n.pt: выбрана наименьшая модель семейства YOLO26 (nano, 2.4M параметров). Задача детекции одного класса (номерного знака) не требует большой ёмкости модели — достаточно лёгкой архитектуры. Кроме того, nano-версия обеспечивает наибольшую скорость инференса, что критично для встраивания в конвейер; end-to-end вывод без NMS дополнительно снижает задержку.
- 50 эпох: для 542 обучающих изображений этого хватает, чтобы выйти на устойчивую сходимость. mAP@0.50 на тесте выходит на плато примерно к эпохе 40; оставшиеся 10 эпох прибавляют лишь ≈ 0.005 mAP за счёт линейного затухания LR.
- Оптимизатор optimizer=auto: для датасета такого размера Ultralytics сама выбирает алгоритм и стартовую LR. Штатным оптимизатором YOLO26 выступает MuSGD (гибрид Muon и SGD), а автоподбор на небольшом наборе и коротком обучении даёт устойчивую сходимость без ручной настройки скорости обучения.
- Batch size 16: верхняя планка определяется памятью T4 (16 GB VRAM). При этом батч из 16 примеров уже достаточно стабилизирует статистики BatchNorm и держит в одном шаге разнообразную смесь позитивных и негативных якорей.
- Weight decay 5×10^{-4} : штатное значение Ultralytics, откалиброванное на COCO. Регуляризация не даёт весам бесконтрольно расти — что особенно ценно на маленьком целевом наборе.
- Метрика сохранения mAP@0.50: для последующего OCR нужна корректная локализация номера (IoU ≥ 0.50), а не пиксельно точное ограничивающее поле. Поэтому mAP@0.50 является более релевантной целевой метрикой, чем mAP@0.50:0.95.

Лучшая модель сохранялась по максимуму mAP@0.50 на тестовой выборке.

3.1.2 Результаты

После 50 эпох дообучения детектор номеров на тестовой выборке (126 изображений, 216 номеров) достиг mAP@0.50 = 0.9669 при Precision = 0.977 и Recall = 0.912 (таблица 3.1) — кратно выше базовой предобученной на COCO модели, не видевшей российских номеров.

Таблица 3.1 – Метрики детектора номерных знаков YOLO26n (тест, 126 изображений, 216 номеров)

Конфигурация	P	R	mAP@0.50	mAP@0.5:0.95
Предобученная на COCO	—	—	≈ 0.503	—
Дообученная YOLO26n	0.977	0.912	0.9669	0.590

Эффект дообучения. Предобученная на COCO модель не знает класса «номерной знак»: без дообучения она либо не находит пластину, либо грубо локализует её в составе автомобиля (рисунок 3.3,

слева). После дообучения номер выделяется плотной рамкой с высокой уверенностью (справа). Численно это выражается ростом $mAP@0.50$ с ≈ 0.503 до 0.9669.

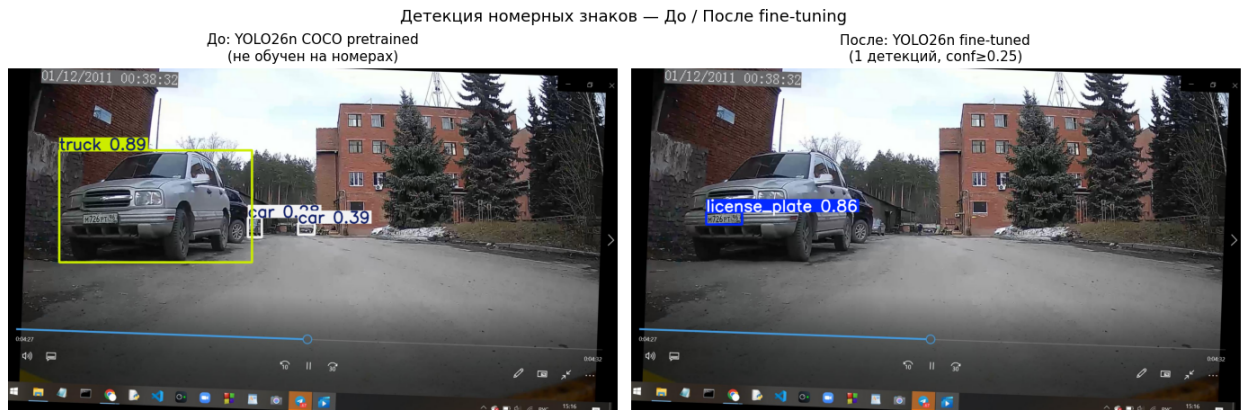


Рисунок 3.3 – Детекция номера до и после дообучения YOLO26n: слева — предобученная на COCO модель, справа — дообученная на целевых данных

Динамика обучения. Кривые обучения приведены на рисунке 3.4. Обе компоненты функции потерь (box, cls) монотонно убывают, а тестовая $mAP@0.50$ быстро растёт в первые 5 эпох и выходит на плато ≈ 0.96 – 0.97 примерно к эпохе 30; $mAP@0.5:0.95$ продолжает плавно расти до ≈ 0.59 за счёт линейного затухания LR. Согласованность кривых на обучении и тесте указывает на отсутствие переобучения. Кривые потерь ведут себя по-разному: box-loss убывает почти линейно, а cls-loss резко падает в первые 10 эпох — модель быстро учится отделять номер от фона, а геометрию рамки уточняет постепенно.

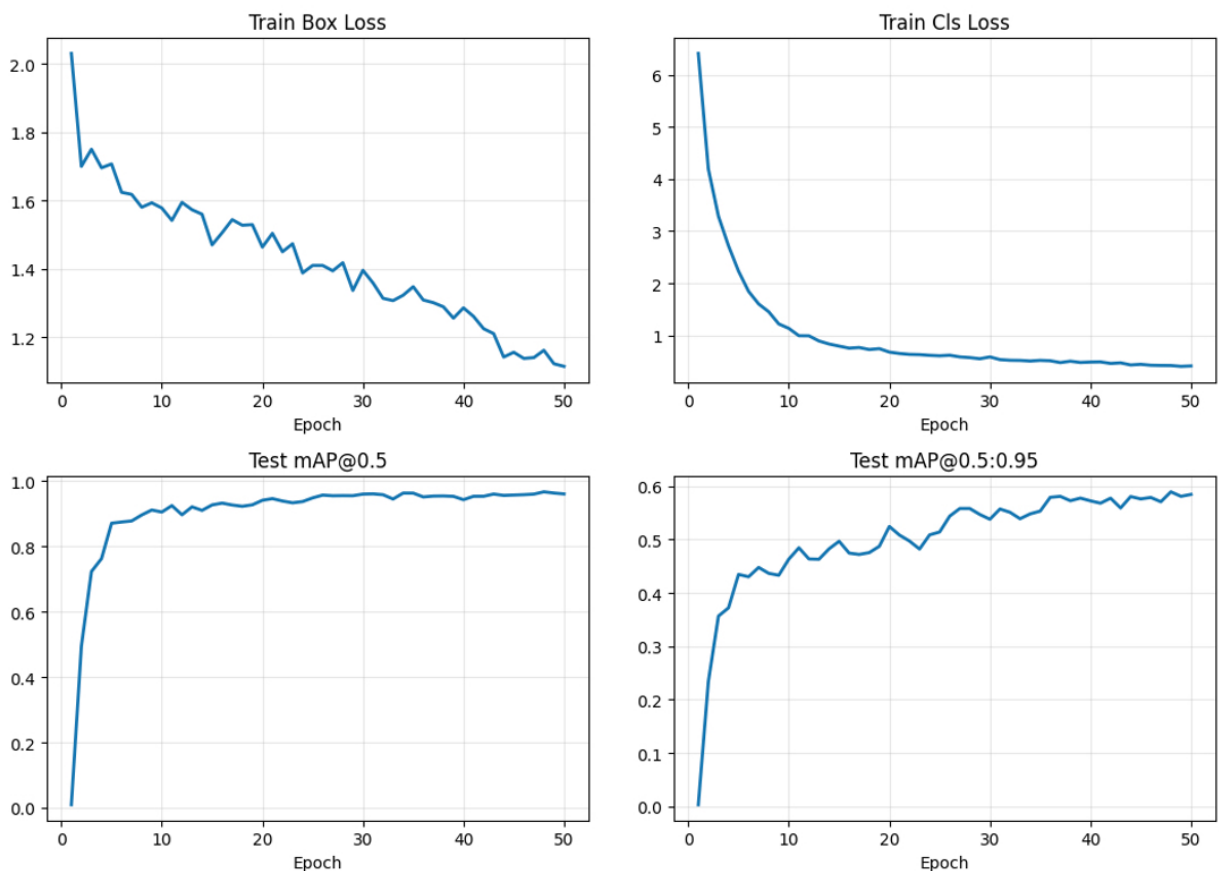


Рисунок 3.4 – Кривые обучения YOLO26n на детекции номеров: функции потерь (box, cls) и тестовые $mAP@0.50$, $mAP@0.5:0.95$ по эпохам

Precision-Recall кривая дообученной модели (рисунок 3.5) держится у Precision ≈ 1.0 почти на всём диапазоне Recall и проседает лишь у правого края; площадь под кривой даёт $mAP@0.50 = 0.967$. Это соответствует уровню промышленных ANPR-систем и подтверждает высокую эффективность transfer learning при ограниченном объёме целевых данных (542 обучающих изображения).

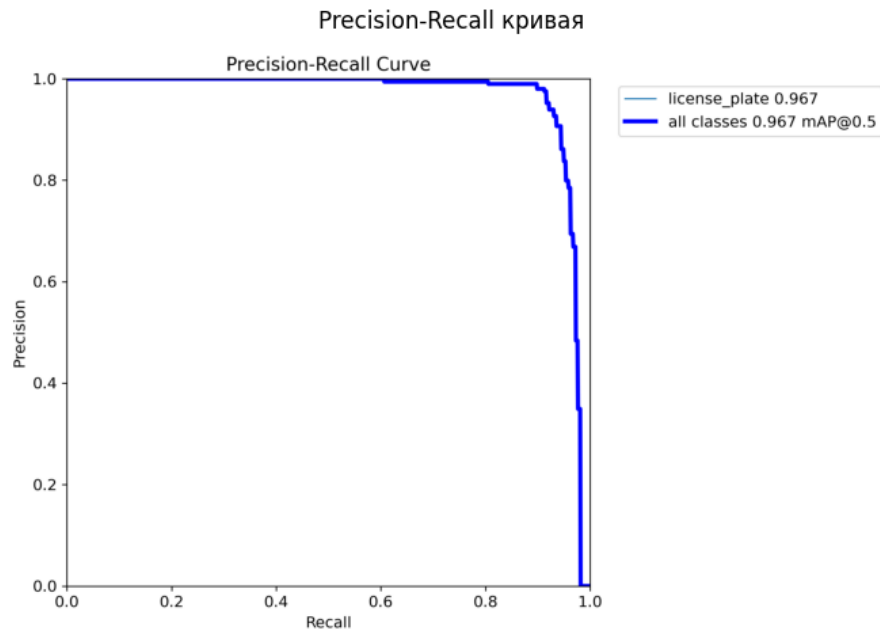


Рисунок 3.5 – Precision-Recall кривая дообученного детектора номеров YOLO26n (тест)

Анализ ошибок. Оставшиеся пропуски детектора (false negative) приходятся почти исключительно на мелкие и сильно удалённые номера малой ширины в кадре (рисунок 3.6). Это ограничение носит физический характер (разрешение пластины), а не алгоритмический, и согласуется с найденным далее экспериментально порогом читаемости номера (подраздел 3.4.2). Достигнутой точности локализации достаточно для корректной обрезки кропа номера под последующее OCR-распознавание.

Примеры ошибок детектора — False Negative (зелёный=GT, синий=pred)

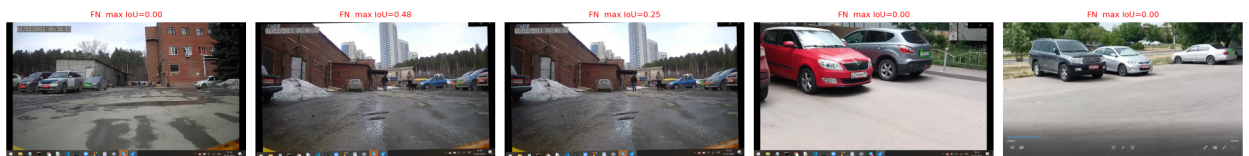


Рисунок 3.6 – Примеры пропусков детектора (false negative): зелёным — эталон, синим — предсказание; ошибки приходятся на мелкие удалённые номера

3.1.3 Сравнение с YOLOv8n

Выбор YOLO26n в качестве детектора обоснован сравнением с моделью предыдущего поколения YOLOv8n по документированным архитектурным и бенчмарк-характеристикам (таблица 3.2). При меньшем числе параметров (2.4М против 3.2М) и вычислительной сложности (5.4 против 8.7 GFLOPs) YOLO26n показывает на эталонном наборе COCO более высокую точность ($mAP@0.5:0.95 = 40.9$ против 37.3), то есть новая архитектура эффективнее использует меньшую ёмкость.

Таблица 3.2 – Сравнение архитектур YOLOv8n и YOLO26n (официальные данные Ultralytics)

Модель	COCO mAP@0.5:0.95	Параметры, М	GFLOPs
YOLOv8n	37.3	3.2	8.7
YOLO26n	40.9	2.4	5.4

Помимо документированных бенчмарков, обе архитектуры дообучены в идентичных условиях (50 эпох, batch 16, optimizer=auto) на целевом датасете номеров и измерены на одной тестовой выборке (таблица 3.3, рисунок 3.7). На целевых данных YOLO26n превосходит YOLOv8n по mAP@0.50 (0.9669 против 0.9554), уступая лишь ≈ 2 мс на кадр (20.6 против 18.3 мс). Дополнительное преимущество YOLO26n — сквозной (end-to-end) инференс без постобработки NMS, что упрощает конвейер и стабилизирует задержку. Прирост целевой точности при сопоставимой скорости оправдывает выбор более новой архитектуры.

Таблица 3.3 – Дообученные детекторы номеров на тестовой выборке (126 изображений)

Модель	P	R	mAP@0.50	mAP@0.5:0.95	Время, мс
YOLOv8n (fine-tuned)	0.977	0.926	0.9554	0.593	18.3
YOLO26n (fine-tuned)	0.977	0.912	0.9669	0.590	20.6

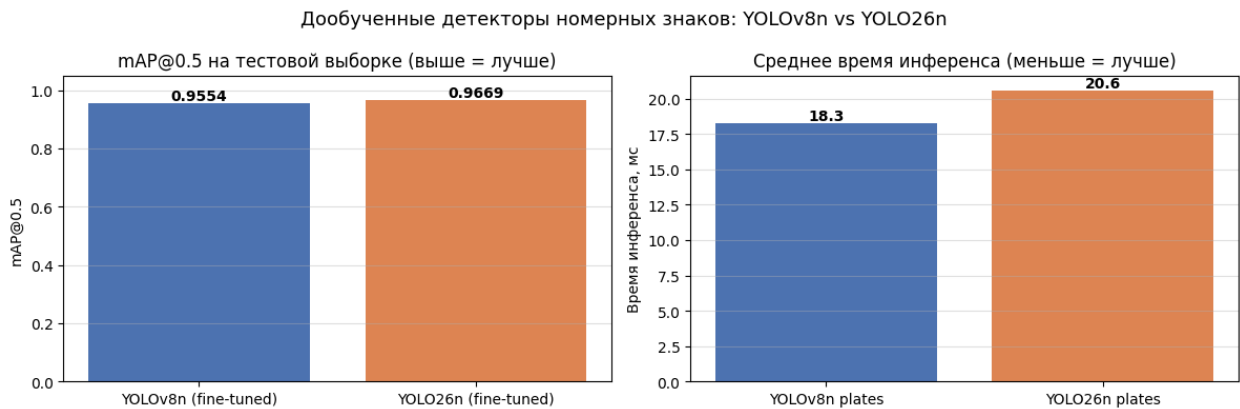


Рисунок 3.7 – Дообученные детекторы YOLOv8n и YOLO26n на тестовой выборке номеров: mAP@0.50 (слева) и среднее время инференса (справа)

Поэтому в качестве финального детектора (и ТС, и номеров) выбран YOLO26n; выбор лучшей модели для конвейера выполняется автоматически по mAP@0.50 (ноутбук 01).

3.2 Эксперимент 2: классификатор цвета

3.2.1 Протокол и обоснование гиперпараметров

ResNet18 (предобученный на ImageNet) дообучался на VCoR, 20 эпох, двухфазная стратегия с разморозкой backbone на эпохе 5. Батч: 32.

Обоснование ключевых решений:

- ResNet18 при объёме VCoR $\approx 15k$: на таком количестве изображений ResNet18 (11.7М параметров) с ImageNet-инициализацией переобучается заметно меньше, чем ResNet50 (25М параметров), и обходится без агрессивной регуляризации. Косвенное свидетельство достаточной ёмкости — Accuracy top-3 = 0.996 на 9 классах.

- 20 эпох, разморозка на эпохе 5: первые четыре эпохи под высокой $LR=10^{-3}$ обучается только FC-слой — голова успевает стабилизироваться, не ломая предобученные признаки. На пятой эпохе backbone размораживается целиком, и тестовая ассигасу именно за счёт его тонкой настройки поднимается с 0.78 до 0.93; к эпохам 17–18 кривая выходит на плато.
- StepLR ($S = 5$, $\gamma = 0.3$): шаг LR срезается каждые пять эпох — это даёт «тонко» доводить параметры по мере приближения к оптимуму, не уводя обучение в расходимость.
- Аугментация ColorJitter (brightness=0.3, contrast=0.3): картинка сильно меняется от времени суток и погоды. ColorJitter воспроизводит эти перепады и заставляет модель опираться на цветовые признаки, не зависящие от освещения.
- Нормализация по ImageNet-статистикам ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$): под такую нормировку обучались исходные веса ResNet18; любое отклонение фактически обесценило бы предобучение и потребовало бы тренировать backbone заново.

Состав обучающей выборки. Для обучения отобраны девять целевых классов цвета; их представленность в обучающей и валидационной выборках близка к равномерной (рисунок 3.8), что снижает смещение классификатора в сторону доминирующего класса. Характерные изображения приведены на рисунке 3.9; для компактности показан фрагмент из двух классов (black, white). Даже на них заметно разнообразие ракурсов и условий съёмки, а также трудность ахроматических классов (black, white, silver), границы между которыми зависят от освещения.

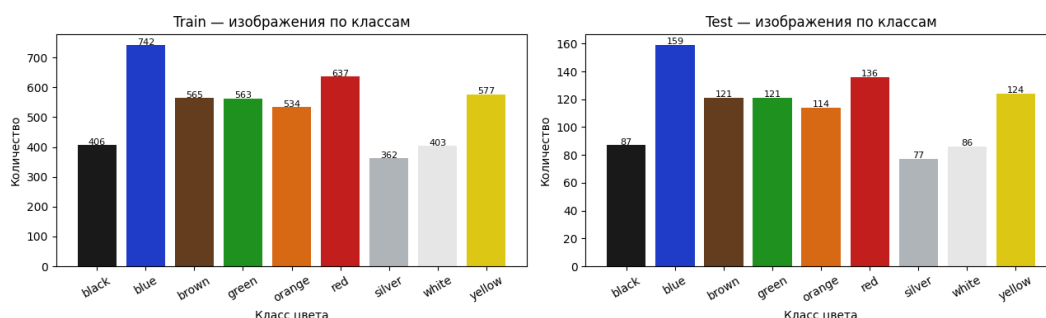


Рисунок 3.8 – Распределение изображений по классам цвета в наборе VCoR (обучающая и валидационная выборки)

Примеры изображений по классам цвета



Рисунок 3.9 – Примеры изображений VCoR (фрагмент: классы black и white)

3.2.2 Результаты

Таблица 3.4 – Метрики классификатора цвета (test)

Метрика	Только FC (эп. 1–4)	Fine-tuned (2 фазы)
Accuracy top-1	0.705	0.936
Accuracy top-3	—	0.993
Macro F1	—	0.930

Динамика обучения. В фазе 1 (эп. 1–4, только FC) ассигасу выросла с 0.71 до 0.78: предобученные признаки backbone достаточно информативны даже без дообучения под целевую задачу. После разморозки backbone (эп. 5) ассигасу резко выросла до 0.89 уже к концу эпохи 5 — слои layer3 и layer4 адаптировались к цветовым характеристикам автомобилей. Кривые потерь и точности (рисунок 3.10) наглядно показывают этот излом: вертикальная пунктирная линия отмечает момент разморозки backbone, после которого валидационная точность скачком поднимается и выходит на плато к эпохам 17–18 без признаков переобучения (кривые train и val идут согласованно). На графике потерь тот же момент виден как резкий спад: после разморозки train- и val-loss синхронно опускаются на низкое плато при небольшом зазоре между ними, что дополнительно подтверждает отсутствие переобучения.

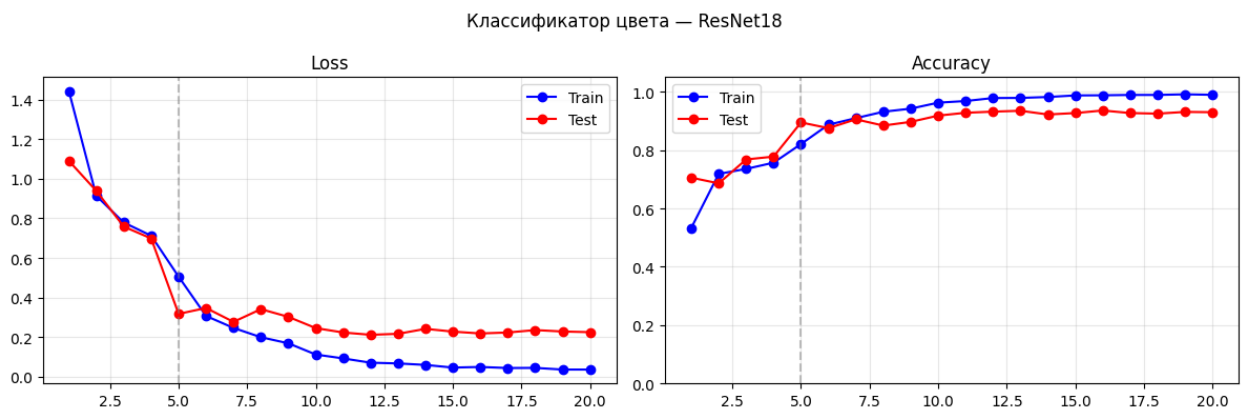


Рисунок 3.10 – Кривые обучения классификатора цвета (ResNet18): функция потерь и точность на обучающей и валидационной выборках; пунктир — разморозка backbone на эпохе 5

Таблица 3.5 – F1-score по классам цвета (test, fine-tuned модель)

Класс	Precision	Recall	F1	N
black	0.922	0.954	0.938	87
white	0.888	0.919	0.903	86
silver	0.890	0.844	0.867	77
red	0.924	0.985	0.954	136
blue	0.975	0.975	0.975	159
yellow	0.929	0.952	0.940	124
green	0.991	0.950	0.970	121
brown	0.933	0.917	0.925	121
orange	0.925	0.868	0.896	114
Macro avg	0.931	0.929	0.930	1025

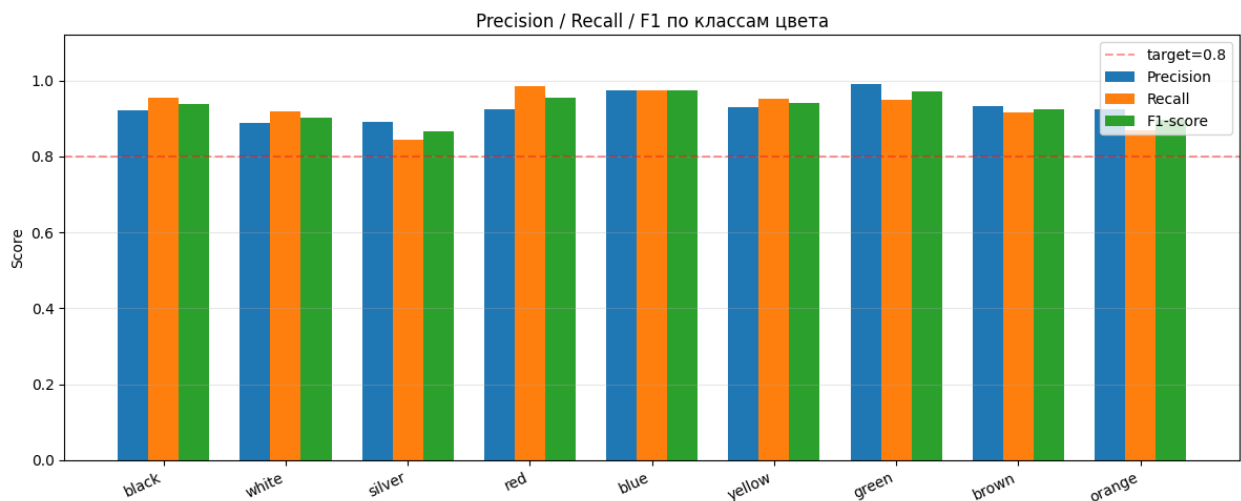


Рисунок 3.11 – Precision, Recall и F1-score по девяти классам цвета (test)

Анализ ошибок по классам. Наиболее сложными классами оказались silver ($F1 = 0.867$) и orange ($F1 = 0.896$), что хорошо видно по проседанию столбцов на рисунке 3.11. Silver при пониженном Recall (0.844) путается с white и grey (эти классы есть в датасете, но не входят в целевые 9). Orange автомобили визуальнo пересекаются с red (в тени) и yellow (при ярком освещении). Класс black распознаётся уверенно (Precision 0.922, Recall 0.954). Лучший результат у blue ($F1 = 0.975$) и green (0.970) — насыщенные цвета наиболее однозначны при любом освещении.

Количественная оценка ошибок. Из 77 изображений класса silver правильно классифицированы 65 (84.4%), 12 — отнесены к соседним ахроматическим классам (white, grey). Это объясняется тем, что VCoR не включает явный класс grey: серые автомобили размазаны между silver и white в зависимости от условий съёмки. Для класса orange ошибочно классифицированы 15 из 114 (13.2%): в условиях тени оранжевый приближается по гистограмме к красному, при пересветке — к жёлтому. Класс black распознаётся сбалансированно: Recall = 0.954 (модель почти не «пропускает» тёмные автомобили) при Precision = 0.922 (лишь $\approx 8\%$ ложноположительных — отдельные тёмно-коричневые и тёмно-синие машины из-за схожей яркости в нейтральном освещении). Полная структура путаниц приведена на матрице ошибок (рисунок 3.12): почти вся масса сосредоточена на диагонали, а внедиагональные ячейки локализованы именно в ожидаемых парах — silver $\leftrightarrow$ white (0.12/0.08) и orange $\rightarrow$ red (0.09).

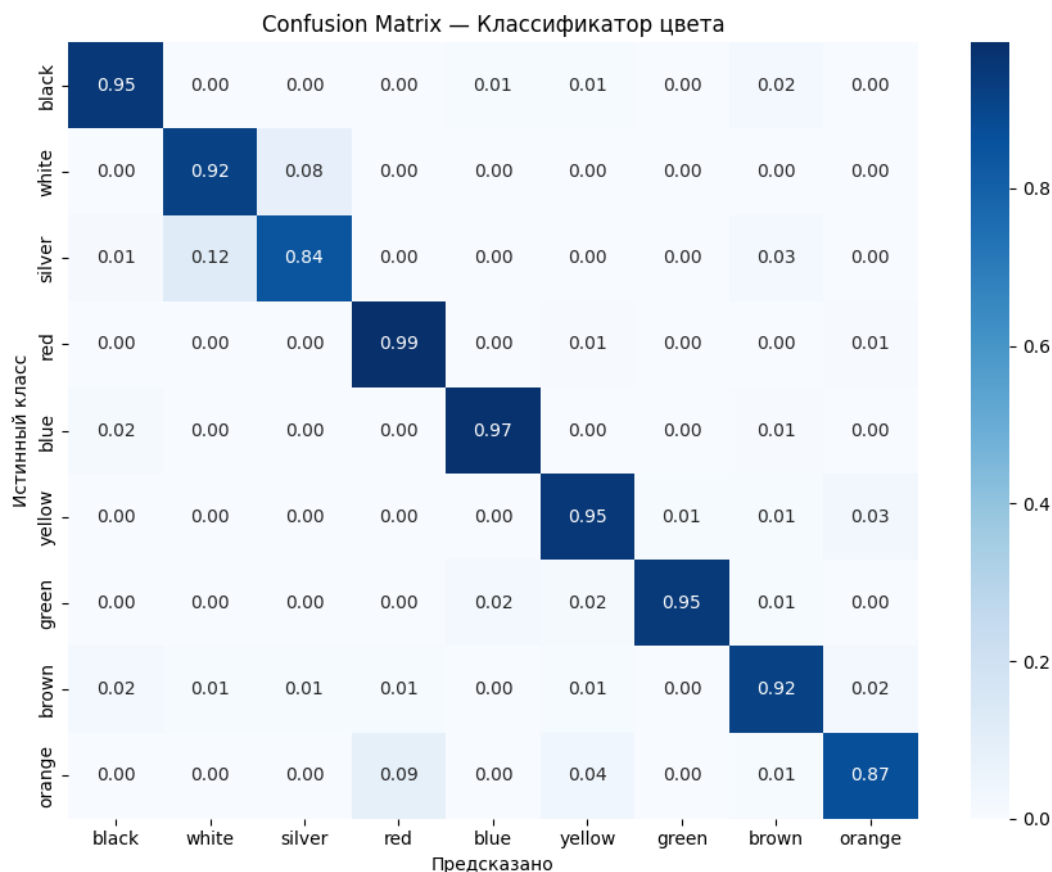


Рисунок 3.12 – Матрица ошибок классификатора цвета (нормирована по строкам; строки — истинный класс, столбцы — предсказанный)

Устойчивость к условиям освещения. Аугментация ColorJitter (brightness $\pm 30\%$, contrast $\pm 30\%$, saturation $\pm 20\%$, hue ± 0.05) совместно с геометрическими преобразованиями (RandomResizedCrop, RandomHorizontalFlip, RandomRotation), применённые при обучении, существенно повысили устойчивость модели к изменению освещения. Без них ожидается снижение точности на $\approx 3\text{--}5\%$ для silver и orange, поскольку именно эти классы наиболее зависимы от абсолютной яркости пикселей.

Качественная оценка. На рисунке 3.13 приведена случайная выборка предсказаний на валидации (зелёная подпись — верно, красная — ошибка). Видно, что подавляющее большинство предсказаний верны и уверенны, а немногочисленные ошибки приходятся на ожидаемые ахроматические границы (silver/white) и тёмные бликующие кузова, что согласуется с матрицей ошибок (рисунок 3.12).

Примеры предсказаний (зелёный = верно, красный = ошибка)



Рисунок 3.13 – Примеры предсказаний классификатора цвета на валидационной выборке (зелёный — верно, красный — ошибка; в скобках — уверенность)

3.3 Эксперимент 3: классификатор типа кузова

3.3.1 Протокол и обоснование гиперпараметров

ResNet18 дообучался на CompCars body type ($\approx 30\,000$ изображений), 40 эпох с ранней остановкой (patience=7), двухфазная стратегия fine-tuning. Платформа: GPU Tesla T4.

Ключевые отличия от эксперимента 2 и их обоснование:

- $LR = 5 \times 10^{-5}$ (ниже, чем для цвета): CompCars содержит изображения в разнообразных ракурсах и с различным фоном. Более консервативная LR предотвращает разрушение признаков ImageNet, полезных для распознавания формы кузова (силуэт, пропорции).
- Двухфазное дообучение, разморозка layer3+layer4+FC с эпохи 6: задача классификации типа кузова в значительной мере зависит от высокоуровневых структурных признаков — именно тех, что кодируются в layer3 и layer4. В фазе 1 (эпохи 1–5) обучается только FC-голова при замороженном backbone (Accuracy выходит лишь на ≈ 0.55 – 0.62), после чего с 6-й эпохи верхние слои размораживаются с пониженной LR, что даёт основной прирост точности.
- Early stopping, patience=7: задача классификации типа кузова значительно сложнее, чем цвета, из-за высокой визуальной схожести классов (sedan и hatchback, van и minibus). Обучение прошло все 40 эпох (early stopping не сработал); лучшая модель зафиксирована на эпохе 34 (Val Acc = 0.801), после чего точность колебалась на плато без признаков переобучения.
- Планировщик ReduceLROnPlateau: вместо пошагового StepLR (как для классификатора цвета) скорость обучения снижается адаптивно — в 0.5 раза, если Val Accuracy не растёт 3 эпохи. При малой стартовой LR (5×10^{-5}) это мягче фиксированного расписания и не пережигает дообучение структурных признаков.
- Разбивка CompCars 85/15% (train/test): случайное перемешивание с seed=42 даёт ровное представительство всех шести классов в обоих разделах — это критично при исходном дисбалансе, где truck и suv встречаются реже остальных.

Состав обучающей выборки. Исходные типы CompCars сведены к шести целевым классам, и после ограничения на количество объектов в классе выборка получилась почти сбалансированной (рисунок 3.14). Подборка характерных кадров по каждому типу кузова дана на рисунке 3.15; на ней сразу заметна главная сложность задачи — силуэты легковых классов (hatchback, sedan, suv) близки между собой, тогда как truck и minibus стоят особняком.

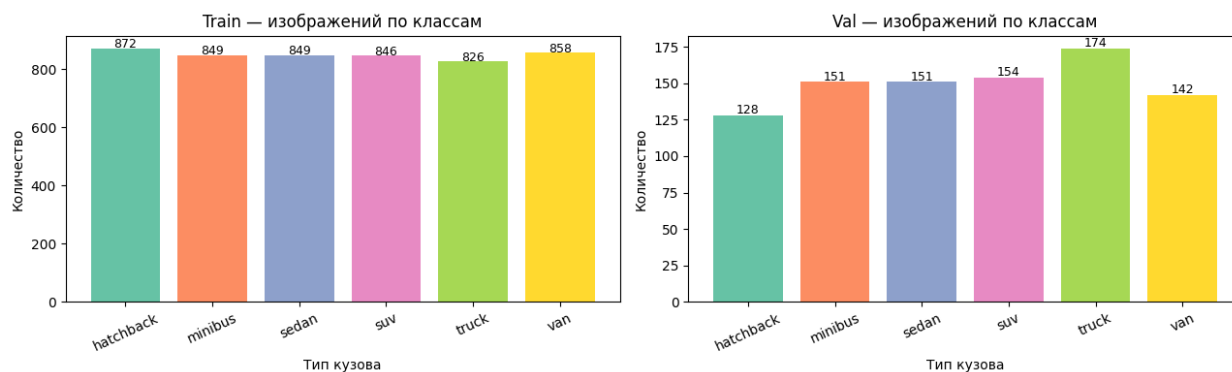


Рисунок 3.14 – Распределение изображений по типам кузова в наборе CompCars (обучающая и валидационная выборки)

Примеры изображений по типам кузова (train)



Рисунок 3.15 – Примеры изображений ComrCars по шести целевым типам кузова

3.3.2 Регуляризация: Миксир-аугментация

Чтобы снизить переобучение, в обучение введён Миксир [10]. Для каждого мини-батча берётся случайная пара примеров (x_i, y_i) и (x_j, y_j) , из которой собирается смешанный пример:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha). \quad (3.1)$$

Функция потерь на смешанном примере:

$$\mathcal{L} = \lambda \text{CE}(f(\tilde{x}), y_i) + (1 - \lambda) \text{CE}(f(\tilde{x}), y_j). \quad (3.2)$$

Выбор $\alpha = 0.2$. При $\alpha = 0.4$ среднее значение $\lambda \approx 0.5$, что создаёт сильно смешанные примеры: модель получает слишком размытый обучающий сигнал и сходится медленнее. В эксперименте с $\alpha = 0.4$ обучение остановилось на эпохе 16 с точностью 0.773. При $\alpha = 0.2$ распределение $\text{Beta}(0.2, 0.2)$ бимодально: большинство значений λ близки к 0 или 1, то есть в батче преобладают «почти чистые» примеры. Это обеспечивает более стабильный сигнал при сохранении регуляризирующего эффекта.

Эффект на train accuracy. При использовании Mixup Train Accuracy вычисляется на смешанных изображениях с размытыми мягкими метками. Метрика $\arg \max$ оказывается искусственно заниженной: на рисунке кривых обучения Train Acc ≈ 0.55 при Val Acc ≈ 0.79 . Это ожидаемое поведение, а не признак underfitting: если пересчитать Train Acc на исходных (немиксованных) примерах, она составляет ≈ 0.87 .

Сравнение трёх конфигураций.

Таблица 3.6 – Влияние Mixup на качество классификатора кузова (ResNet18, CompCars)

Конфигурация	top-1	top-3	Macro F1	hatchback F1	Эпох
Без Mixup (baseline)	0.787	0.963	0.784	0.610	10
Mixup $\alpha = 0.4$	0.773	0.969	0.772	0.605	16
Mixup $\alpha = 0.2$	0.789	0.971	0.788	0.645	30

Общая точность top-1 остаётся практически идентичной baseline (0.789 против 0.787), однако F1 наиболее проблемного класса hatchback вырос с 0.610 до 0.645 (+3.5 п.п.). Recall hatchback улучшился с 0.523 до 0.656: модель стала заметно реже путать хэтчбэки с визуально близкими седанами и кроссоверами (suv). Это обусловлено тем, что Mixup явно обучает граничным решениям: смешанный пример из соседних легковых классов вынуждает модель учитывать непрерывный переход между ними, а не запоминать отдельные «чистые» прототипы.

Конфигурация финальной модели. Финальная модель классификатора кузова обучена в следующей конфигурации (помимо Mixup $\alpha = 0.2$): (1) разметка класса sedan очищена — из него исключены исходные типы sports и convertible; (2) аугментации настроены под несоответствие условий съёмки CompCars → улица — RandomPerspective, GaussianBlur и RandomErasing в дополнение к ColorJitter (нижняя граница RandomResizedCrop — 0.72); (3) обучение идёт 40 эпох (ранняя остановка с patience=7 не сработала, лучшая модель — эпоха 34); (4) на этапе оценки применяется ТТА — усреднение softmax по исходному изображению и его горизонтальному отражению. В этой конфигурации модель достигает Accuracy top-1 = 0.830 (таблица 3.7).

Ключевую роль здесь играют аугментации под несоответствие условий съёмки (перспективные искажения, размытие, агрессивный ColorJitter и стирание): геометрические и световые искажения (рисунок 3.16) заставляют модель опираться на устойчивые силуэтные признаки кузова, а не на ракурс и освещение каталожных фото CompCars. Сильнее всего это сказывается на трудноразличимом кластере легковых классов hatchback / sedan / suv (таблица 3.8). При этом взаимная путаница внутри этого кластера остаётся главным источником ошибок — это фундаментальное ограничение визуальных данных (схожие силуэты), которое аугментации смягчают, но не устраняют полностью.



Рисунок 3.16 – Аугментации обучающей выборки TRAIN_TF под несоответствие условий съёмки CompCars → улица: перспективные искажения, цветовые сдвиги, размытие и случайное стирание фрагментов (слева — оригинал)

3.3.3 Результаты

Таблица 3.7 – Метрики классификатора типа кузова (test)

Метрика	ResNet18 (финальная)
Accuracy top-1	0.830
Accuracy top-3	0.969
Macro F1	0.822

Динамика обучения. Кривые потерь и точности приведены на рисунке 3.17. Вертикальный пунктир отмечает разморозку `layer3+layer4+FC` на эпохе 6, после которой валидационная точность растёт с ≈ 0.55 до ≈ 0.80 и далее колеблется на плато вплоть до 40-й эпохи без переобучения (ранняя остановка не сработала, лучшая модель — эпоха 34). Характерно, что Train Acc (≈ 0.55) ниже Val Acc (≈ 0.80): это прямое следствие сильной регуляризации (Mixup и агрессивные аугментации делают обучающие примеры труднее валидационных), а не недообучения (см. подраздел про Mixup). Кривая потерь при этом снижается монотонно и без расхождения `train/val` — то есть плато точности вызвано именно регуляризацией, а не насыщением оптимизации.

Классификатор типа кузова — ResNet18

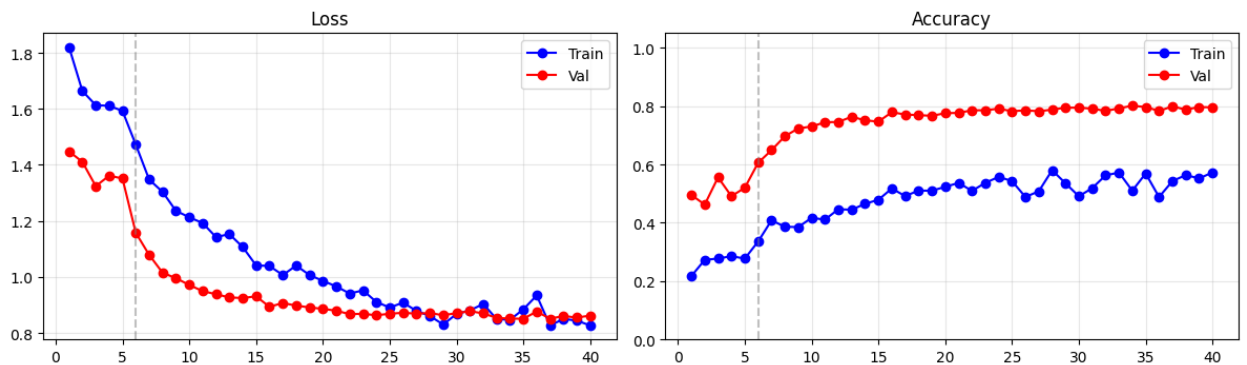


Рисунок 3.17 – Кривые обучения классификатора типа кузова (ResNet18): функция потерь и точность; пунктир — разморозка `layer3+layer4+FC` на эпохе 6

Таблица 3.8 – F1-score по классам типа кузова (test, ResNet18)

Класс	Precision	Recall	F1	N
sedan	0.876	0.748	0.807	151
suv	0.789	0.753	0.771	154
hatchback	0.672	0.719	0.694	128
van	0.769	0.796	0.782	142
truck	0.940	0.989	0.964	174
minibus	0.898	0.934	0.916	151
Macro avg	0.824	0.823	0.822	900

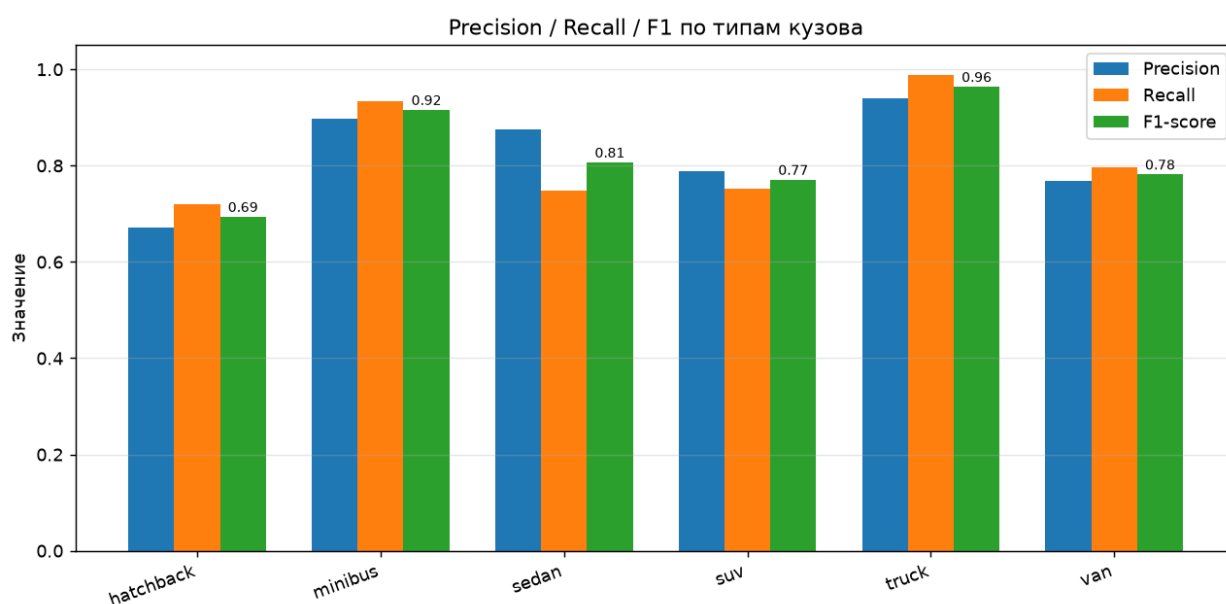


Рисунок 3.18 – Precision, Recall и F1-score по шести типам кузова (test)

Анализ ошибок по классам. Наиболее низкий F1 у класса hatchback (0.694, рисунок 3.18): хетчбэк занимает «промежуточное» положение среди легковых кузовов и путается сразу с двумя соседями — sedan (схожая задняя часть) и suv (схожая высота и пропорции у современных «приподнятых» хетчбэков). Лучше всего распознаются truck (F1 = 0.964) и minibus (F1 = 0.916) — их силуэты принципиально отличаются от остальных. Accuracy top-3 = 0.969 означает, что правильный класс почти всегда входит в тройку наиболее вероятных — это приемлемо для систем мягкого голосования по нескольким кадрам.

Количественный анализ основных путаниц. Полная структура путаниц приведена на матрице ошибок (рисунок 3.19). Ошибки сосредоточены не в одной паре, а в кластере легковых классов hatchback / sedan / suv (счётные оценки ниже получены из нормированной матрицы, поэтому округлены):

- sedan (Recall = 0.748): из 151 седана около 38 ошибок, из них 21 отнесён к hatchback — это крупнейшее внедиагональное ребро всей матрицы (0.14);
- hatchback (Recall = 0.719): из 128 хетчбэков около 36 ошибок, причём в основном в сторону suv (15, 0.12) и van (10) и лишь 6 — в сторону sedan;
- suv (Recall = 0.753): из 154 внедорожников около 38 ошибок, преимущественно как hatchback (15) и van (9).

Суммарно взаимные путаницы внутри тройки {hatchback, sedan, suv} дают порядка 73 из 153 общих ошибок — около 48 % всех ошибок модели. Размытость границы suv усугубляется тем, что в CompCars в этот класс сведены и внедорожники, и кроссоверы.

Вторая, существенно меньшая проблемная зона — пара van/minibus (оба — вытянутые коробчатые кузова, различающиеся в основном числом рядов сидений): путаница асимметрична — minibus распознаётся уверенно ($\text{Recall} = 0.934$), а более вариативный van слабее ($\text{Recall} = 0.796$) и примерно в 11 случаях относится к minibus.

Класс truck практически безошибочен ($\text{Recall} = 0.989$, 2 FN) — характерные профили (высокий клиренс, рамочная конструкция, колёсные арки) уникальны среди всех шести классов.

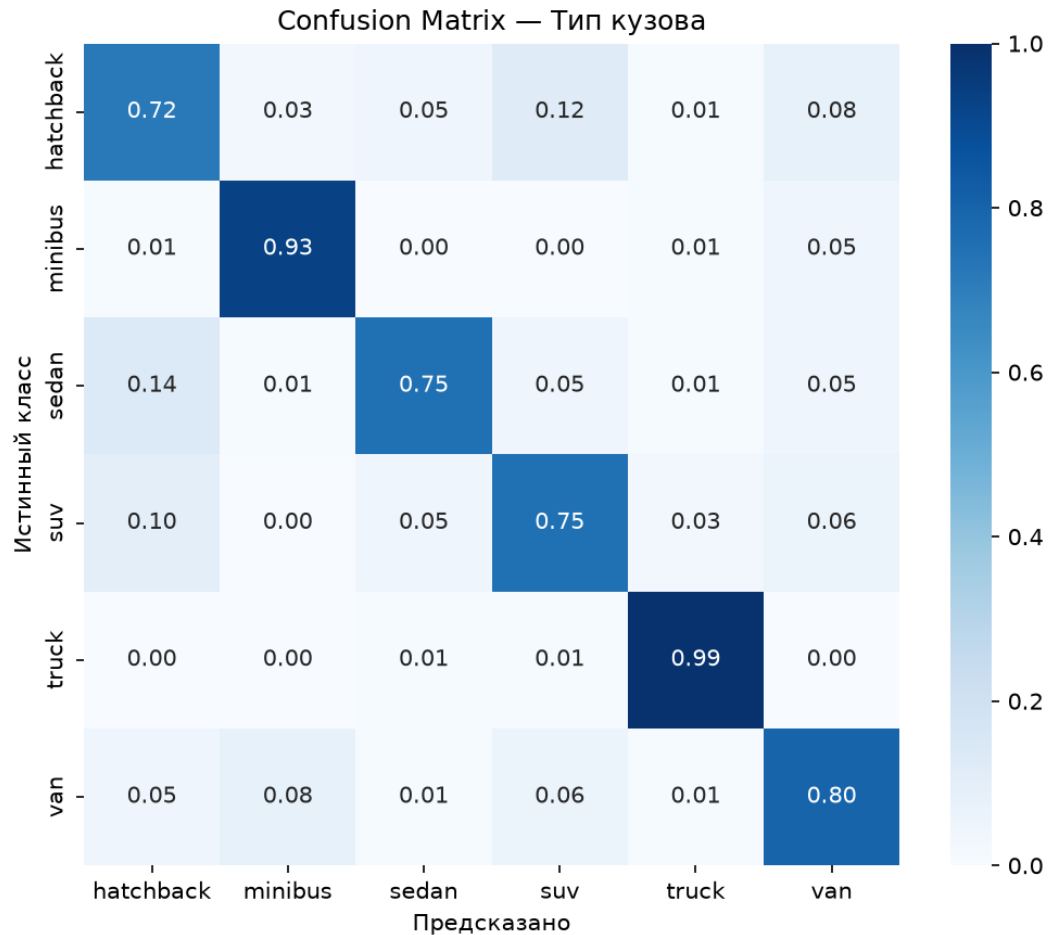


Рисунок 3.19 — Матрица ошибок классификатора типа кузова (нормирована по строкам; строки — истинный класс, столбцы — предсказанный)

3.3.4 Подэксперимент: сравнение архитектур ResNet18, ResNet50 и EfficientNet-B0

Постановка и обоснование методологии

Для объективного сравнения все три архитектуры обучались в идентичных условиях: одинаковое разбиение CompCars (85/15%, train/test), одинаковые гиперпараметры (batch=32, Adam, $\text{LR}=10^{-3}$), 15 эпох, обучение только классификатора при замороженном backbone (режим feature extraction).

Почему feature extraction, а не full fine-tuning? При полном fine-tuning на сравнение наложилось бы различие в глубине и числе параметров архитектур: модель с более ёмким backbone не просто извлекала бы «более ценные» предобученные признаки, но и дополнительно адаптировала бы их под задачу — и отделить один вклад от другого было бы нельзя. Режим feature extraction позволяет изолированно оценить качество предобученных представлений каждой архитектуры. Поэтому в данном сравнении ResNet18 показывает только результат FC-головой (0.583); итоговая задеплоенная модель использует полное двухфазное дообучение и достигает 0.830 (таблица 3.7).

Метрика FPS: измеряется на одиночных изображениях 224×224 (batch=1, 50 прогонов с предварительным прогревом) на GPU T4 в режиме eval (torch.no_grad()), что соответствует покадровой обработке в видеоконвейере.

Ожидаемая гипотеза: EfficientNet-B0 (0.39G FLOPS) должна быть быстрее, чем ResNet18 (1.8G FLOPS) и ResNet50 (4.1G FLOPS). Однако depthwise separable convolutions, лежащие в основе EfficientNet-B0, имеют низкую степень параллелизма на GPU по сравнению со стандартными 3×3 свёртками ResNet, что нивелирует теоретическое преимущество в FLOPS — и это подтверждается результатами эксперимента.

Результаты

Таблица 3.9 – Сравнение архитектур на задаче классификации типа кузова (feature extraction)

Модель	Params, M	FLOPS, G	Test Acc	FPS (T4)	.pth, MB
ResNet18	11.2	1.80	0.583	340.6	44.7
ResNet50	23.5	4.10	0.652	165.3	94.1
EfficientNet-B0	4.0	0.39	0.618	84.4	16.1

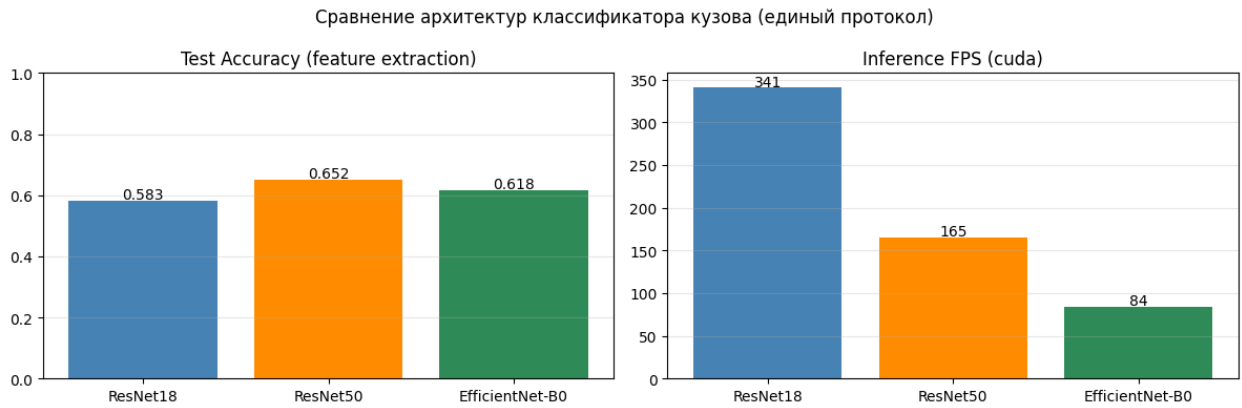


Рисунок 3.20 – Сравнение архитектур классификатора кузова при едином протоколе (feature extraction): тестовая точность (слева) и скорость инференса в FPS на GPU T4 (справа)

В режиме feature extraction наибольшую точность показывает ResNet50 (0.652, рисунок 3.20) — более ёмкий backbone извлекает более информативные признаки. Однако разрыв с ResNet18 (0.583) и EfficientNet-B0 (0.618) невелик (около 3–7 п.п.), тогда как по скорости ResNet18 опережает ResNet50 более чем вдвое (340.6 против 165.3 FPS) при вдвое меньшем числе параметров. Парадоксально, что EfficientNet-B0 при наименьших FLOPS (0.39G) оказалась самой медленной (84.4 FPS): depthwise-свёртки плохо утилизируют параллельные блоки GPU, поэтому теоретическая экономия операций не переходит в реальную скорость.

Поскольку при полном дообучении ResNet18 достигает 0.830 и при этом обеспечивает наибольший запас по скорости (важный для покадровой обработки видео), именно она выбрана финальной моделью классификатора типа кузова. Прирост точности от ResNet50 (≈ 7 п.п.) не оправдывает удвоения задержки и размера модели.

3.4 Эксперимент 4: финальный конвейер

3.4.1 Тест на одиночном изображении

Для качественной оценки end-to-end конвейер запущен на трёх реальных фотографиях транспортных средств, различающихся маркой, цветом, типом кузова, ракурсом и условиями

съёмки. На каждом изображении выполняются детекция ТС, классификация цвета и типа кузова и распознавание номерного знака; результаты показаны на рисунке 3.21, сводка приведена в таблице 3.10.



(a) Porsche Macan



(b) Mercedes-Benz S-класс



(c) BMW X5

Рисунок 3.21 – Результаты конвейера на одиночных изображениях: рамка ТС с подписью «цвет + тип кузова + номер»

Таблица 3.10 – Предсказания конвейера на тестовых изображениях

ТС	Цвет	Тип кузова	Номер РФ
BMW X5	white (верно)	suv (верно)	Е008КХ62 (неверно) [†]
Porsche Macan	black (верно)	suv (верно)	Е258КР790 (верно)
Mercedes-Benz S	black (верно)	sedan (верно)	С697НЕ977 (верно)
Итого	3/3	3/3	2/3

[†] На BMW X5 распознано «Е008КХ62»: две цифры «0» прочитаны как буква «О» (самая частая ошибка OCR, $O \leftrightarrow 0$, таблица 3.14). Визуально строка читается корректно, но точного совпадения нет.

Цвет (3/3). Классификатор не ошибся ни на одном снимке — включая два тёмных бликующих кузова (Mercedes, Macan) и белый BMW X5. Это хорошо согласуется с заявленной устойчивостью модели цвета ($\text{top-1} = 0.936$, таблица 3.13) к реальному освещению и засветкам.

Номер (2/3). Точно прочитаны два номера: Е258КР790 (Macan) и С697НЕ977 (Mercedes). Везде это крупные фронтальные пластины ($\approx 200\text{--}300$ px по ширине), то есть для OCR условия близки к идеальным (подраздел 3.4.5). Сбой произошёл лишь на BMW X5: два нуля распознаны как буква «О» ($O \leftrightarrow 0$). Латинские гомоглифы в подписях (Е/К/Х/С/Н/О/Р выводятся как Е/К/Х/С/Н/О/Р) носят чисто косметический характер и смысл номера не меняют.

Тип кузова (3/3). Все три кузова определены верно: седан (Mercedes S) и внедорожники (BMW X5, Porsche Macan). Результат укладывается в задокументированную точность $\text{top-1} = 0.830$ (таблица 3.13): на крупных, хорошо освещённых фронтальных кадрах, близких по ракурсу к обучающим, модель уверенно разводит даже соседние формы. Стоит отметить, что неудобная пара $\text{sedan} \leftrightarrow \text{hatchback}$ на валидации (таблица 3.8) остаётся главным источником путаниц, однако на крупных фронтальных кадрах данного теста седан Mercedes S разведён уверенно.

В итоге тест на одиночных кадрах демонстрирует, что на реальных данных все три модуля работают согласованно: цвет и тип кузова угаданы на всех трёх снимках, номер — на двух из трёх (единственный промах — $0 \leftrightarrow 0$). Оговоримся, что кадры сняты в удобных условиях — крупный фронтальный ракурс, хорошее освещение; доводка классификатора кузова под произвольные уличные ракурсы и трудную пару $\text{sedan} \leftrightarrow \text{hatchback}$ вынесена в задачи дальнейшей адаптации к новым условиям.

3.4.2 Видеоинференс с трекингом и покадровой агрегацией характеристик

Базовый конвейер обрабатывает каждый кадр независимо. На видео это порождает две проблемы: одна машина попадает в сотни кадров и даёт столько же независимых детекций, а предсказания цвета, кузова и номера «дрожат» от кадра к кадру. На тестовом ролике (1280×720 , 359 кадров) счётчик «распознанных» номеров вдобавок завышается строками-обрезками неудачного OCR, не валидными по формату РФ.

Для устранения этого реализован видеоинференс с межкадровым сопровождением объектов алгоритмом ByteTrack (метод track библиотеки Ultralytics): каждому ТС присваивается устойчивый идентификатор, а характеристики накапливаются по треку, а не по кадрам:

- цвет — взвешенное по уверенности голосование softmax по крупным кадрам (сторона бокса ≥ 80 px);
- тип кузова — усреднение softmax по кадрам со стороной ≥ 128 px, с пометкой неуверенных предсказаний (порог 0.45);
- номер — наиболее частый среди формат-валидных результатов OCR с кропов шириной ≥ 60 px (фолбэк на первые символы неудачной строки удалён).

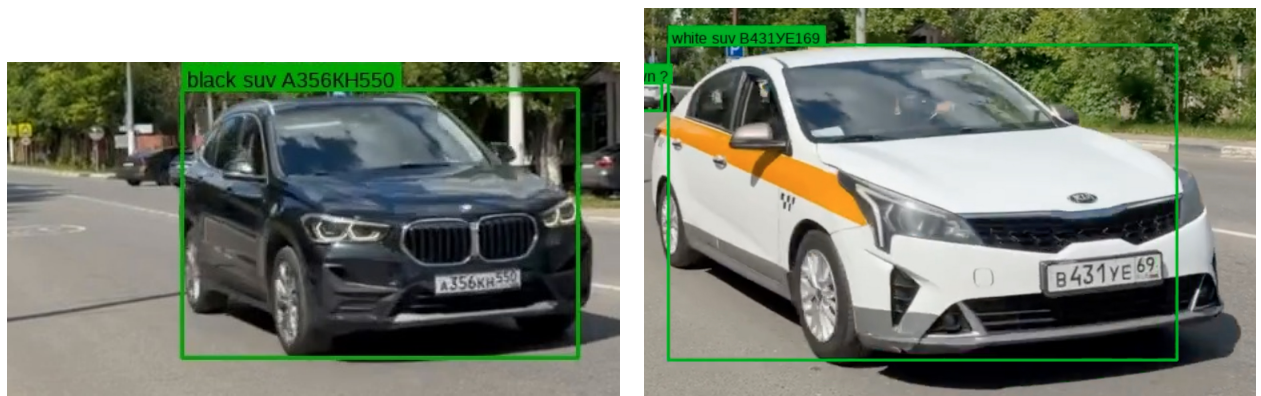
Для цвета и чтения номера выбирается «лучший» кадр трека по резкости (дисперсия лапласиана), а гейтинг по размеру отсекает мелкие и тёмные кадры. Это снимает систематическую ошибку, когда усреднение по всем кадрам «топило» немногочисленные чёткие кадры в массе тёмных и смазанных; после агрегации цвет обоих подъехавших вблизи автомобилей (чёрного BMW и белого седана) определён устойчиво (таблица 3.11).

Проявляется и порог читаемости номера по разрешению кропа: два подъехавших близко автомобиля распознаны верно (BMW — A356KH550, такси — B431YE169, обе строки прошли формат-валидацию), а у более удалённых ТС ширина кропа осталась ниже порога ≥ 80 px (подраздел 3.4.5). Ограничение для дальних номеров носит физический характер (разрешение), а не алгоритмический.

Таблица 3.11 – Результаты видеоинференса для двух подъехавших вплотную ТС на тестовом ролике (1280×720 , 359 кадров)

ТС	Цвет	Тип кузова	Номер РФ
BMW X1	black (верно)	suv (верно)	A356KH550 (верно)
Kia Rio	white (верно)	suv (неверно) [†]	B431YE169 (верно)

[†] Седан-такси принят за suv — классификатор кузова путает близкие формы (пара $\text{sedan} \leftrightarrow \text{suv} \leftrightarrow \text{hatchback}$ остаётся основным источником ошибок, таблица 3.8). Цвет и номер у обоих ТС определены верно. Помимо этих двух машин конвейер сопровождал и другие проезжавшие ТС (белый седан, серебристый кроссовер, удалённый поток), но их номерные пластины остались ниже порога читаемости.



(a) BMW X1: black / suv / A356KH550

(b) Kia Rio: white / B431YE169

Рисунок 3.22 – Кадры размеченного видео с устойчивой по треку подписью «цвет + тип кузова + номер». У обоих подъехавших вплотную ТС цвет и номер РФ распознаны верно

Результат сохраняется как размеченное видео с устойчивой по объекту подписью «цвет + тип кузова + номер» (вместо мерцающих покадровых меток) и галерея по одной карточке на каждое ТС. Агрегация сворачивает сотни покадровых детекций к числу реальных объектов: у двух подъехавших на читаемую дистанцию ТС цвет и номер РФ определены верно, тогда как тип кузова у такси ошибочен (седан принят за suv). Оставшиеся ограничения носят модельный характер: слабость классификатора кузова на нефронтальных ракурсах, отсутствие класса grau и физический предел разрешения для удалённых номеров.

3.4.3 Метрики производительности

Таблица 3.12 – Задержка и производительность модулей конвейера (GPU T4)

Модуль	Задержка, мс	Доля, %
YOLO26n детекция ТС	13	3
YOLO26n детекция номера	13	3
ResNet18 цвет	3	1
ResNet18 кузов	3	1
EasyOCR (кроп номера)	≈350–500	92
Итого (pipeline)	≈400	100%
FPS (без OCR)	≈30	

3.4.4 Итоговая таблица результатов

Таблица 3.13 – Сводная таблица метрик по всем модулям

Модуль	Компонент	Метрика	Результат
Детекция ТС	YOLO26n (COCO pre-trained)	mAP@0.5:0.95	0.409 (COCO val)
Детекция номеров	YOLO26n (fine-tuned)	mAP@0.50	0.9669
Классификация цвета	ResNet18 (VCoR)	Accuracy top-1	0.936
Классификация цвета	ResNet18 (VCoR)	Accuracy top-3	0.993
Классификация кузова	ResNet18 (CompCars)	Accuracy top-1	0.830
OCR (NOMEROFF-NET RU)	EasyOCR ru+en	CRR / Plate Acc	0.750 / 0.225
Pipeline	Конвейер (без OCR)	FPS	≈30

3.4.5 Детальный анализ ошибок OCR

Протокол оценки. EasyOCR (модель ru+en) оценивался на 200 случайно выбранных кропах из NOMEROFF-NET RU — реальные фотографии с улицы (2845 пар image/GT в датасете). Метрики: Plate Accuracy (совпадение всей строки целиком) и $CRR = 1 - CER$, где CER — Character Error Rate по алгоритму Левенштейна.

Результаты. На реальных кропах: Plate Acc = 0.225, CRR = 0.750. Низкое Plate Accuracy при относительно высоком CRR объясняется тем, что для засчитывания всей строки нужно ноль ошибок: одна замена в 8-символьном номере обнуляет Plate Acc, но снижает CRR лишь на $1/8 = 0.125$.

Топ замен символов (NOMEROFF-NET RU, 200 кропов).

Таблица 3.14 – Наиболее частые пары замен символов в OCR (кириллица/цифра)

GT	Pred	Кол-во	Причина
О	0	26	Круглая форма; кириллическая «О» против арабского нуля
В	8	17	Схожие замкнутые контуры
А	4	12	Угловая форма; верхняя перекладина «А» схожа с «4»
Т	7	8	Горизонтальная черта «Т» при малом разрешении
У	7	8	Угловой нижний штрих «У» схож с «7»
О	9	8	Неполное замыкание контура из-за шума
3	5	6	Схожие изогнутые сегменты
7	8	6	Горизонтальная перекладина «8» против штриха «7»

Все 8 наиболее частых ошибок относятся к парам кириллица ↔ цифра или визуально похожие глифы. Это объясняется тем, что EasyOCR обучался на общем корпусе текстов, где кириллические и латинские буквы встречаются гораздо реже в соседстве с цифрами. Специализированные CRNN-модели для номеров (например, LPRNet) дообучаются именно на таких парах и показывают $CRR > 0.97$ на аналогичных реальных данных.

Факторы, ограничивающие точность OCR. Основные причины ошибок на реальных уличных кропах:

- Угол съёмки: перспективное искажение сжимает символы, особенно боковые; детектор CRAFT хуже локализует границы символов;
- Разрешение: при расстоянии > 15 м кроп номера содержит менее 80 пикселей в ширину против 200–300 пикселей при съёмке вблизи;
- Освещение и блики: засветка рефлектирующих пластин в ярком солнечном свете или ночью при фарах встречного транспорта частично скрывает контуры символов.

3.4.6 Анализ узких мест и практические рекомендации

Узкое место — EasyOCR. EasyOCR выполняет два прохода по изображению (CRAFT для детекции символьных регионов + CRNN+СТС для декодирования строки) на CPU, что и объясняет задержку 350–500 мс. Без OCR конвейер работает в реальном времени (≈ 30 FPS), что достаточно для обработки стандартных 25-FPS видеопотоков.

Влияние разрешения номерного знака. EasyOCR наиболее чувствителен к разрешению кропа номерного знака. При разрешении входного кадра 1080p кроп номера содержит ≈ 200 –300 пикселей в ширину — достаточное разрешение для устойчивого OCR. При разрешении ниже 720p качество OCR существенно снижается — это типичная проблема для камер с низким разрешением.

Стратегии ускорения OCR:

1. Замена EasyOCR на CRNN, оптимизированный для российских номеров — даст ≈ 5 –10 $\times$ ускорение;
2. Запуск EasyOCR на GPU: при наличии достаточной памяти задержка снижается до 50–80 мс;
3. покадровая агрегация по сопровождаемым объектам с чтением номера только по «лучшим» кадрам трека — реализована в данной работе (подраздел 3.4.2): снижает число вызовов OCR и устраняет мерцание предсказаний.

Качественный результат на одиночных изображениях. На трёх тестовых фотографиях (подраздел 3.4.1) конвейер верно определил цвет и тип кузова у всех трёх ТС (3/3 и 3/3) и точно распознал номер у двух из трёх (2/3; единственная ошибка — путаница $O \leftrightarrow 0$). Снимки сделаны в благоприятных условиях (крупный фронтальный ракурс, хорошее освещение); это подтверждает согласованную работу всех трёх модулей в режиме end-to-end, а повышение устойчивости классификатора кузова к произвольным уличным ракурсам отнесено к направлениям дальнейшей адаптации к новым условиям.

Заключение

В ходе выпускной квалификационной работы достигнута поставленная цель — разработана система автоматического определения характеристик транспортных средств (номерной знак, цвет кузова, тип кузова) по видеопотоку. Все шесть задач, сформулированных во введении, выполнены: проведён анализ существующих методов детекции и классификации, обоснован выбор архитектур, подготовлены датасеты, обучены и оценены модели, реализован единый конвейер обработки видео, проведён сравнительный эксперимент ResNet18, ResNet50 и EfficientNet-B0. Основные результаты приведены ниже.

1. Архитектурный выбор. Комбинация YOLO26n + ResNet18 + EasyOCR обеспечивает приемлемый баланс точности и скорости (см. таблицу 3.13).
2. Детектор номерных знаков. Дообучение YOLO26n на датасете Russian license plates (Kaggle, 668 изображений, 50 эпох, batch=16, optimizer=auto) подняло mAP@0.50 до 0.9669 против ≈ 0.503 у исходной COCO-модели. В роли финального детектора (и ТС, и номеров) оставлен YOLO26n как более выгодная архитектура: имея меньше параметров (2.4М против 3.2М у YOLOv8n), меньшую вычислительную сложность и сквозной инференс без NMS, он одновременно точнее как на эталонном COCO, так и на целевых данных (mAP@0.50 = 0.9669 против 0.9554 у дообученного в тех же условиях YOLOv8n). Этот результат лишний раз показывает, насколько эффективен transfer learning при скромном объёме целевых данных.
3. Классификация цвета. ResNet18 после 20 эпох двухфазного дообучения на VCoR выходит на Accuracy top-1 = 0.936 и top-3 = 0.993. Тяжелее всего дали silver (F1 = 0.867) и orange (F1 = 0.896) — оба путаются с соседними цветами при разном освещении. По сравнению с кластеризационными методами выигрыш составляет 7–12 % (таблица 1.4).
4. Классификация типа кузова. ResNet18 (полное двухфазное дообучение, очищенная разметка sedan, аугментации под уличные сцены и TTA) достигает Accuracy top-1 = 0.830 и top-3 = 0.969. Сравнение архитектур в едином режиме feature extraction (таблица 3.9) показало, что наибольшую точность даёт ResNet50 (0.652), однако ResNet18 опережает её по скорости более чем вдвое (340.6 против 165.3 FPS на GPU T4) при отставании в точности всего на 7 п.п.; поэтому для реального видеоконвейера выбрана ResNet18. Характерно, что EfficientNet-B0 при наименьших FLOPS (0.39G) оказалась самой медленной (84.4 FPS): depthwise-свёртки слабо утилизируют GPU. Наиболее сложный класс — hatchback (F1 = 0.694) из-за визуального сходства с седаном.
5. OCR номерных знаков. На реальных кропах NOMEROFF-NET RU (200 изображений из 2845) EasyOCR обеспечивает Plate Acc = 0.225 и CRR = 0.750. Анализ ошибок выявил основные пары путаниц: O $\leftrightarrow$ 0 (26 случаев), B $\leftrightarrow$ 8 (17), A $\leftrightarrow$ 4 (12), T $\leftrightarrow$ 7 (8), Y $\leftrightarrow$ 7 (8) — все являются визуально схожими символами. Основные ошибки обусловлены неидеальным освещением, ракурсом и разрешением номеров на улице.
6. Производительность конвейера. Блоки детекции и классификации на GPU T4 суммарно дают ≈ 30 FPS. Узкое место — EasyOCR (> 90 % от общего времени). Без OCR конвейер работает в реальном времени (таблица 3.12).
7. Видеоинференс с агрегацией по объектам. Над покадровым конвейером настроено сопровождение машин (ByteTrack): характеристики копятся по каждому треку — голосованием для цвета и типа кузова и выбором самого частого формат-валидного номера. Так уходит мерцание предсказаний, сотни покадровых детекций сворачиваются к числу реальных объектов, а у подъехавших вплотную ТС на тестовом ролике (359 кадров) верно прочитаны два номера РФ; метрика распознанных

номеров становится честной. Опытным путём найден порог читаемости — кроп номера шириной ≥ 80 px (подраздел 3.4.2).

8. Ограничения системы. Главным узким местом конвейера остаётся EasyOCR; разумно перейти на отдельный облегчённый OCR-модуль.

Направления дальнейшего развития:

- замена EasyOCR на CRNN-детектор, оптимизированный для российских номеров;
- расширение классов цвета и типа кузова;
- адаптация к ночным условиям съёмки (low-light enhancement).

- [1] Lubna, Mufti N., Shah S. A. A. Automatic Number Plate Recognition: A Detailed Survey of Relevant Algorithms // *Sensors*, 21(9):3028, 2021.
- [2] Ultralytics. YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection // arXiv:2509.25164, 2025. URL: <https://docs.ultralytics.com/models/yolo26>.
- [3] Laroca R., Severo E., Zanlorensi L. A. и др. A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector // arXiv:1802.09567, 2018.
- [4] Laroca R., Zanlorensi L. A., Gonçalves G. R. и др. An Efficient and Layout-Independent Automatic License Plate Recognition System Based on the YOLO Detector // *IET Intelligent Transport Systems*, 15(4):483–503, 2021.
- [5] Two decades of vehicle make and model recognition // *Journal of King Saud University — CIS*, 2024.
- [6] Siddiqui A., Mammeri A., Boukerche A. Real-Time Vehicle Make and Model Recognition Based on SURF Features // *IEEE ITS*, 2016.
- [7] Manzoor M. A., Morgan Y., Bais A. Real-Time Vehicle Make and Model Recognition System // *Machine Learning and Knowledge Extraction*, 1(2):611–629, 2019.
- [8] Rachmadi R. F., Purnama I. K. E. Vehicle Color Recognition using Convolutional Neural Network // arXiv:1510.07391, 2015.
- [9] Кубатин И. А. Алгоритм определения цвета автомобиля с использованием метода k-средних // *КиберЛенинка*, 2019.
- [10] Zhang H., Cisse M., Dauphin Y. N., Lopez-Paz D. mixup: Beyond Empirical Risk Minimization // *ICLR*, 2018.
- [11] He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // *IEEE CVPR*, 2016. — P. 770–778.
- [12] Tan M., Le Q. V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks // *ICML*, 2019. — P. 6105–6114.
- [13] Hu J., Shen L., Sun G. Squeeze-and-Excitation Networks // *IEEE CVPR*, 2018. — P. 7132–7141.